PROMETHEUS_&_GRAFANA

1.Install Prometheus on a server and configure prometheus.yml.

Prerequisites

- One Linux EC2 instance.
- SSH access with sudo privileges.
- In the EC2 Security Group, allow TCP port 9090 from your trusted IP address to access the Prometheus web UI.

Step 1: Update the server and install prerequisites

Connect to your EC2 instance and execute:

sudo dnf update -y

sudo dnf install -y wget tar

Verify the operating system:

cat /etc/os-release

Step 2: Download and install Prometheus

Create a dedicated Linux user and directories:

sudo useradd --no-create-home --shell /sbin/nologin prometheus

sudo mkdir -p /etc/prometheus

sudo mkdir -p /var/lib/prometheus

Download and extract Prometheus:

cd /tmp

wget https://github.com/prometheus/prometheus/releases/latest/download/prometheus-*.linux-amd64.tar.gz

If the wildcard download does not work, download the latest Linux AMD64 archive from the official Prometheus downloads page.

Extract the downloaded archive:

tar -xzf prometheus-*.linux-amd64.tar.gz

cd prometheus-*.linux-amd64

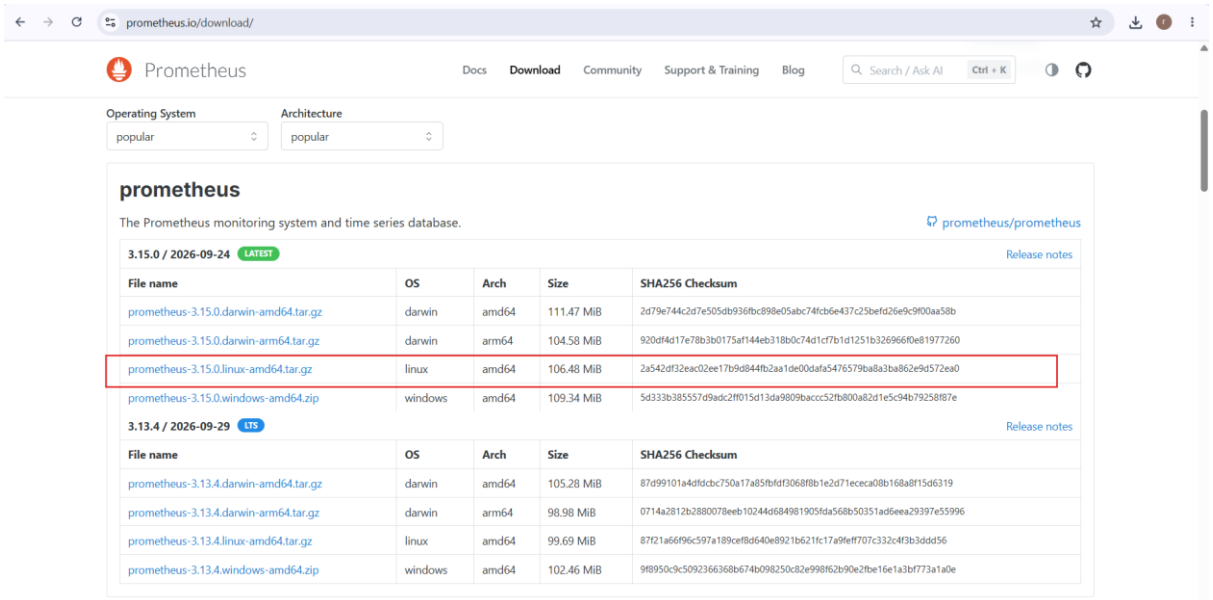
Copy the binaries:

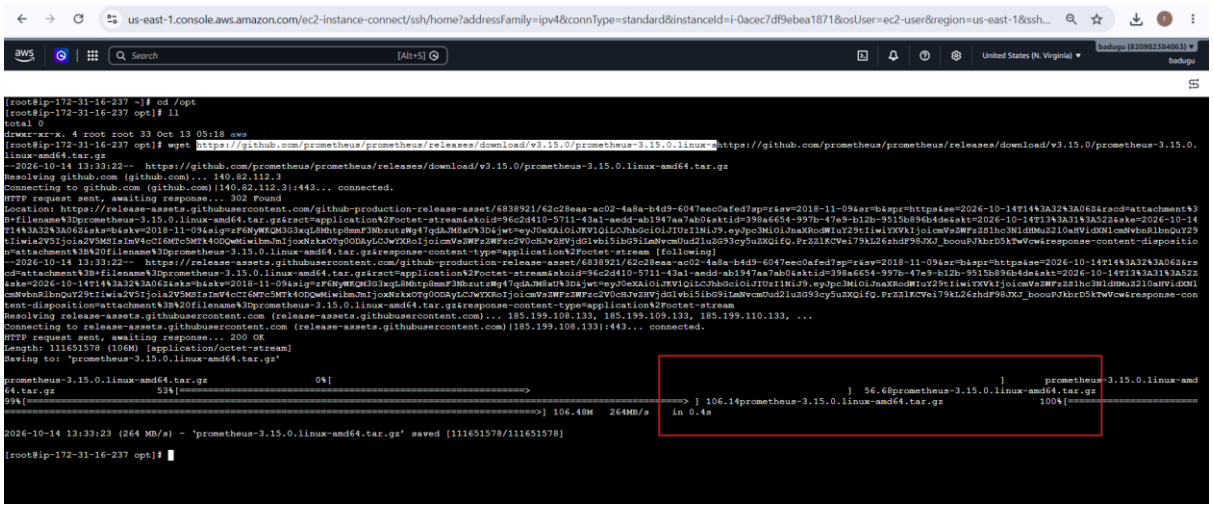
sudo cp prometheus /usr/local/bin/

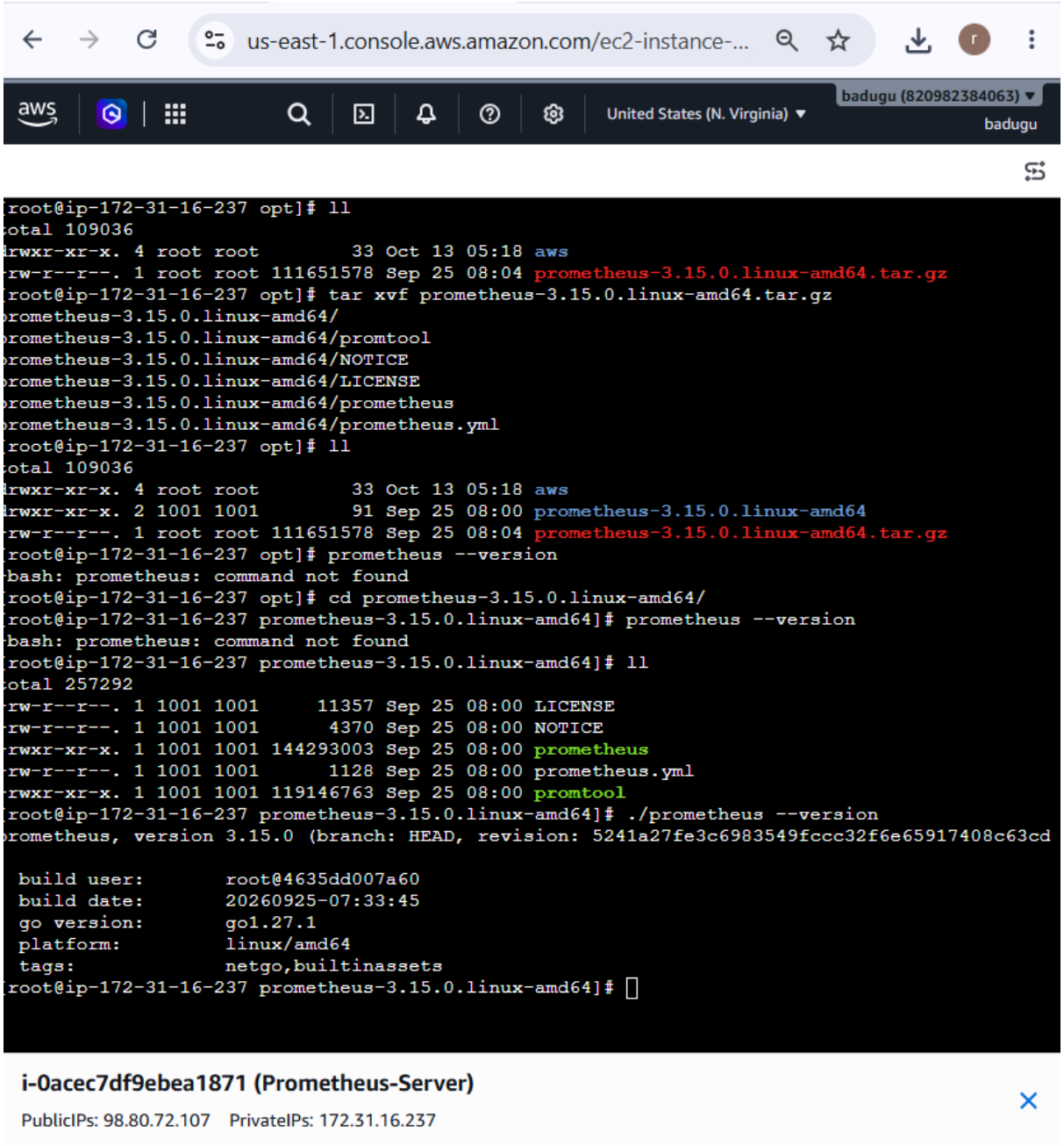
sudo cp promtool /usr/local/bin/

Verify the installation:

prometheus --version




us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-0acec7df9ebea1871&osUser=ec2-user&region=us-east-1&ssh...
[root@ip-172-31-16-237 ~]# cd /opt
[root@ip-172-31-16-237 opt]# ll
total 0
drwxr-xr-x. 4 root root 33 Oct 13 05:18 aws
[root@ip-172-31-16-237 opt]# wget https://github.com/prometheus/prometheus/releases/download/v3.15.0/prometheus-3.15.0.linux-amd64.tar.gz
--2026-10-14 13:33:22-- https://github.com/prometheus/prometheus/releases/download/v3.15.0/prometheus-3.15.0.linux-amd64.tar.gz
Resolving github.com (github.com)... 140.82.112.3
Connecting to github.com (github.com)|140.82.112.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/6838921/62c28eaa-ac02-4a8a-b4d9-6047eec0afed?sp=r&sv=2018-11-09&sr=b&spr=https&se=2026-10-14T14%3A32%3A06Z&rscd=attachment%3B+filename%3Dprometheus-3.15.0.linux-amd64.tar.gz&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-10-14T13%3A31%3A52Z&ske=2026-10-14T14%3A32%3A06Z&sks=b&skv=2018-11-09&sig=rF6NyWKQM3G3xqL8Mhtp8mmF3NbzutzWg47qdAJM8x0%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoicmVvc2FzZS1hc3N1dHMuc2210aHVidXN1cmNvbnRlbnQuY29tIiwia2V5IjoiazF5MHIzImV4cCI6MTc5MTk4ODQwMiwibmJmIjoxNzkxOTg0ODAyLCJwYXRoIjoicmVvc2FzZS1hc3N1dHMtcHJvZHVjdGlvbi1yZWxlYXNlLzY4Mzg5MjIxL2NvbnRlbnQtZGlzcG9zaXRpb249YXR0YWNobWVudCUzQiUyMGZpbGVuYW1lJTNEcHJvbWV0aGV1cy0zLjE1LjAubGludXgtYW1kNjQudGFyLmd6JnJzY3Q9YXBwbGljYXRpb24lMkZvY3RldC1zdHJlYW0mc2tvaWQ9OTZjMjQxMGEtNTcxMS00M2ExLWFlZGQtYWIxOTQ3YWE3YWIwJnNrdGlkPTM5OGE2NjU0LTk5N2ItNDdlOS1iMTJiLTk1MTVCODk2YjREJnNrdD0yMDI2LTEwLTE0VDEzJTNBMzElM0E1Mlomc2tlPTIwMjYtMTAtMTRUMTQlM0EzMiUzQTA2WiZzaWc9cUZ5eTNzSmRzSlVlSlFRNlVlSlFRNlVlSlFRN



Step 3: Configure prometheus.yml

Create the configuration file:

sudo vi /etc/prometheus/prometheus.yml

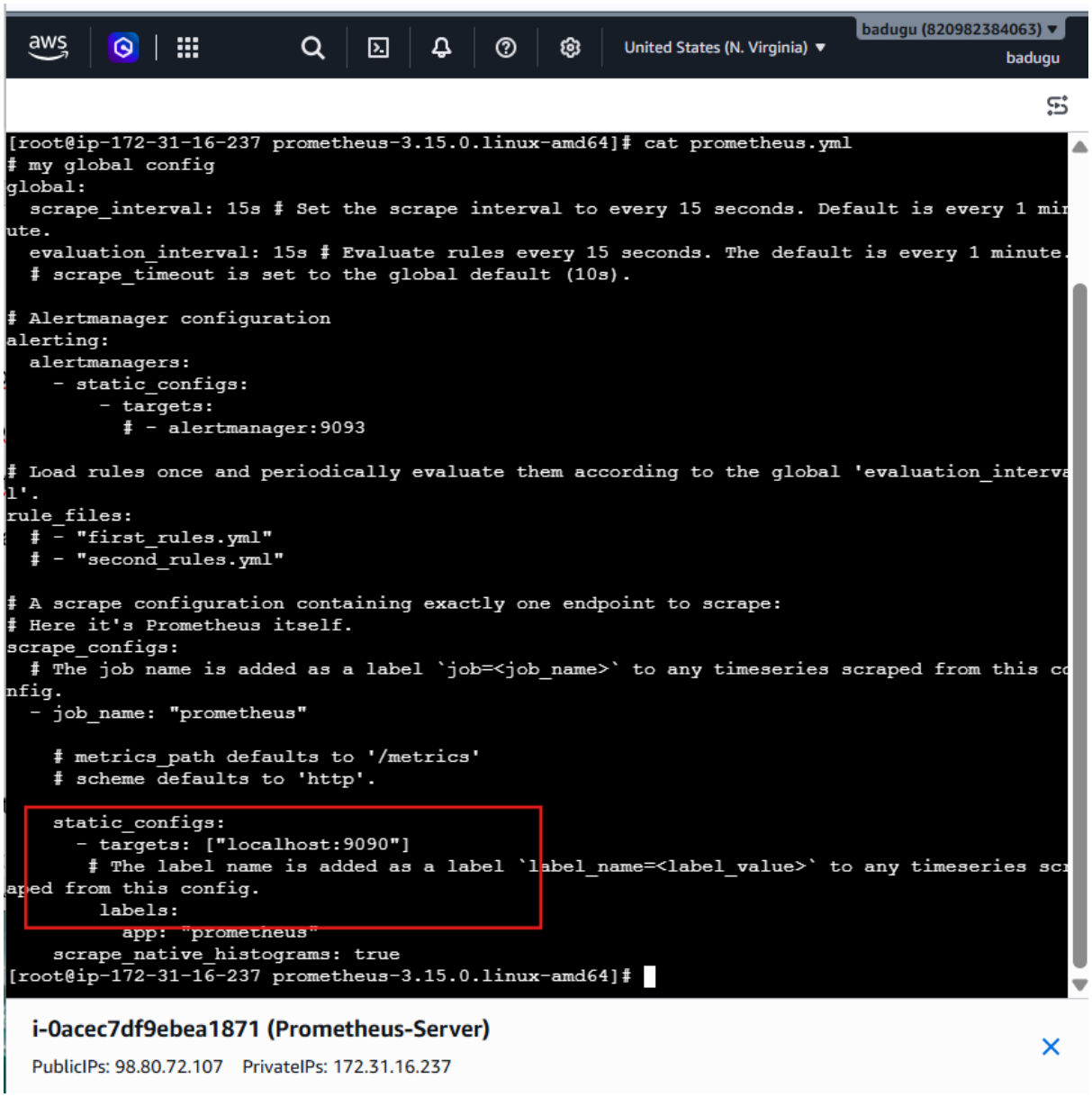
Add the following configuration:

global:

scrape_interval: 15s

evaluation_interval: 15s

```
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```



Configuration explained:

- scrape_interval: 15s — collects metrics every 15 seconds.
- evaluation_interval: 15s — evaluates alerting and recording rules every 15 seconds.

- job_name: prometheus — identifies the Prometheus monitoring job.
- targets: ["localhost:9090"] — tells Prometheus to monitor its own metrics endpoint.

Validate the configuration:

sudo /usr/local/bin/promtool check config /etc/prometheus/prometheus.yml

Expected result: SUCCESS.

Step 4: Set permissions

Give the Prometheus user access to its binaries, configuration, and data directory:

sudo chown prometheus:prometheus /usr/local/bin/prometheus

sudo chown prometheus:prometheus /usr/local/bin/promtool

sudo chown -R prometheus:prometheus /etc/prometheus

sudo chown -R prometheus:prometheus /var/lib/prometheus

Verify permissions:

ls -l /usr/local/bin/prometheus

ls -ld /etc/prometheus /var/lib/prometheus

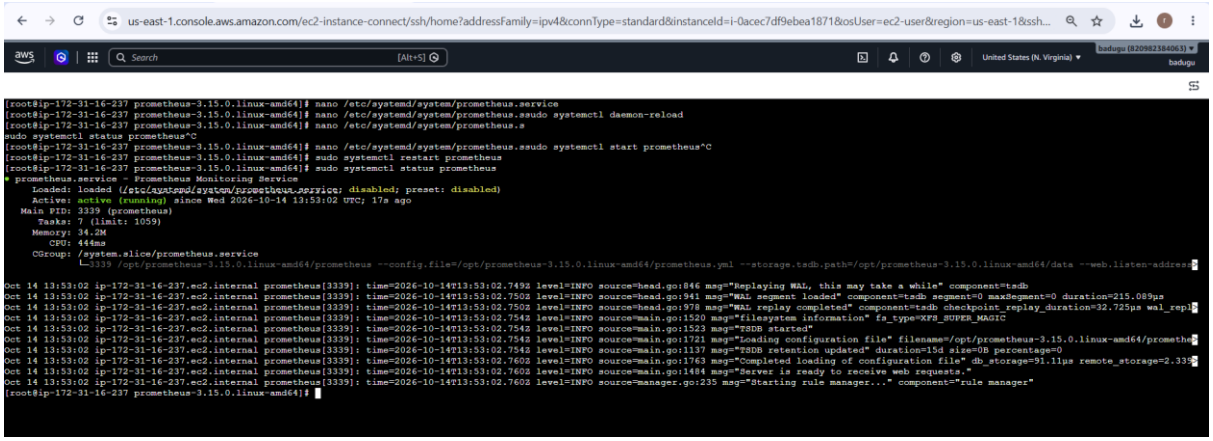
Step 5: Create a systemd service

Create the service file:

sudo vi /etc/systemd/system/prometheus.service




```
[root@ip-172-31-16-237 prometheus-3.15.0.linux-amd64]# cat /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus Monitoring Service
Wants=network-online.target
After=network-online.target
[Service]
User=root
Group=root
Type=simple
ExecStart=/opt/prometheus-3.15.0.linux-amd64/prometheus \
  --config.file=/opt/prometheus-3.15.0.linux-amd64/prometheus.yml \
  --storage.tsdb.path=/opt/prometheus-3.15.0.linux-amd64/data \
  --web.listen-address=:9090
Restart=on-failure
[Install]
WantedBy=multi-user.target
[root@ip-172-31-16-237 prometheus-3.15.0.linux-amd64]#
```



Step 6: Verify Prometheus in the browser

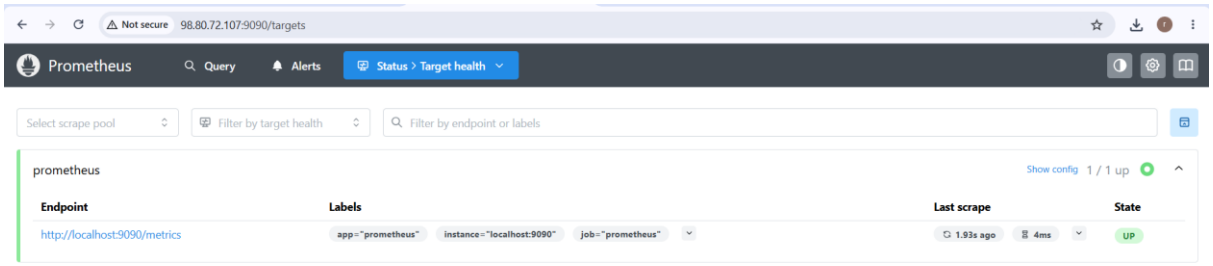
Check the local endpoint first:

curl http://localhost:9090/-/healthy

Expected output:

Prometheus Server is Healthy.

Find your EC2 public IP in the AWS EC2 console and open this address in your browser:

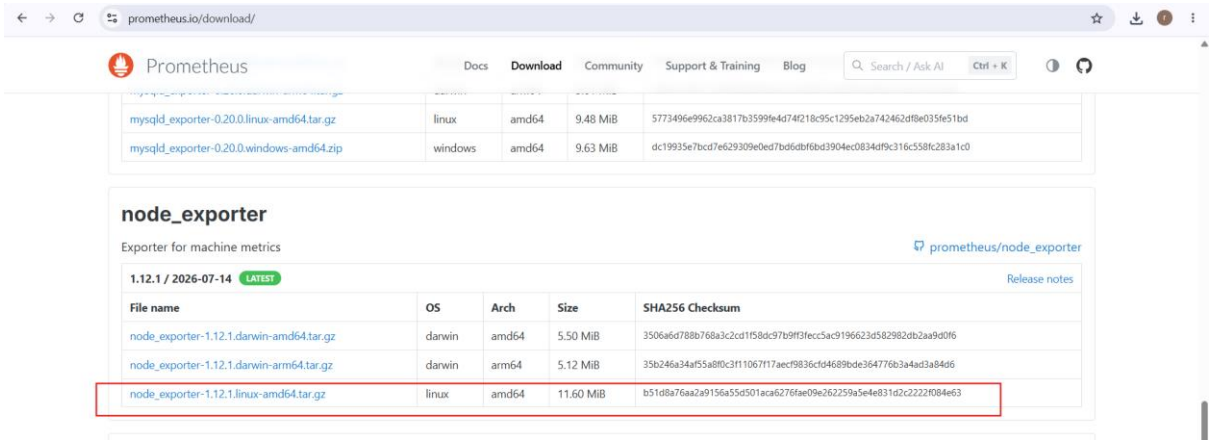


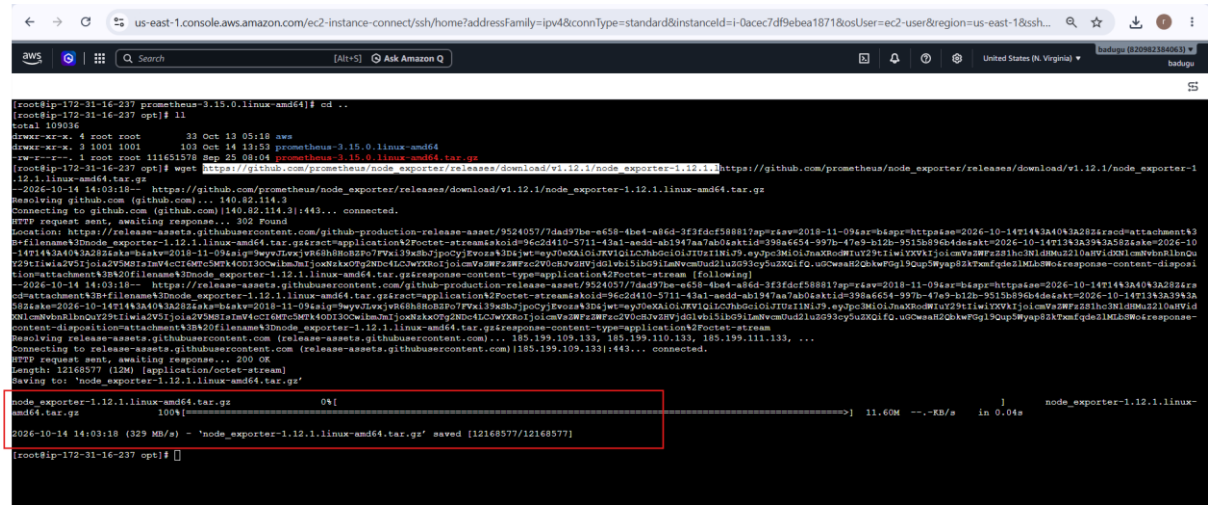
2.Install and configure Node Exporter for system monitoring.

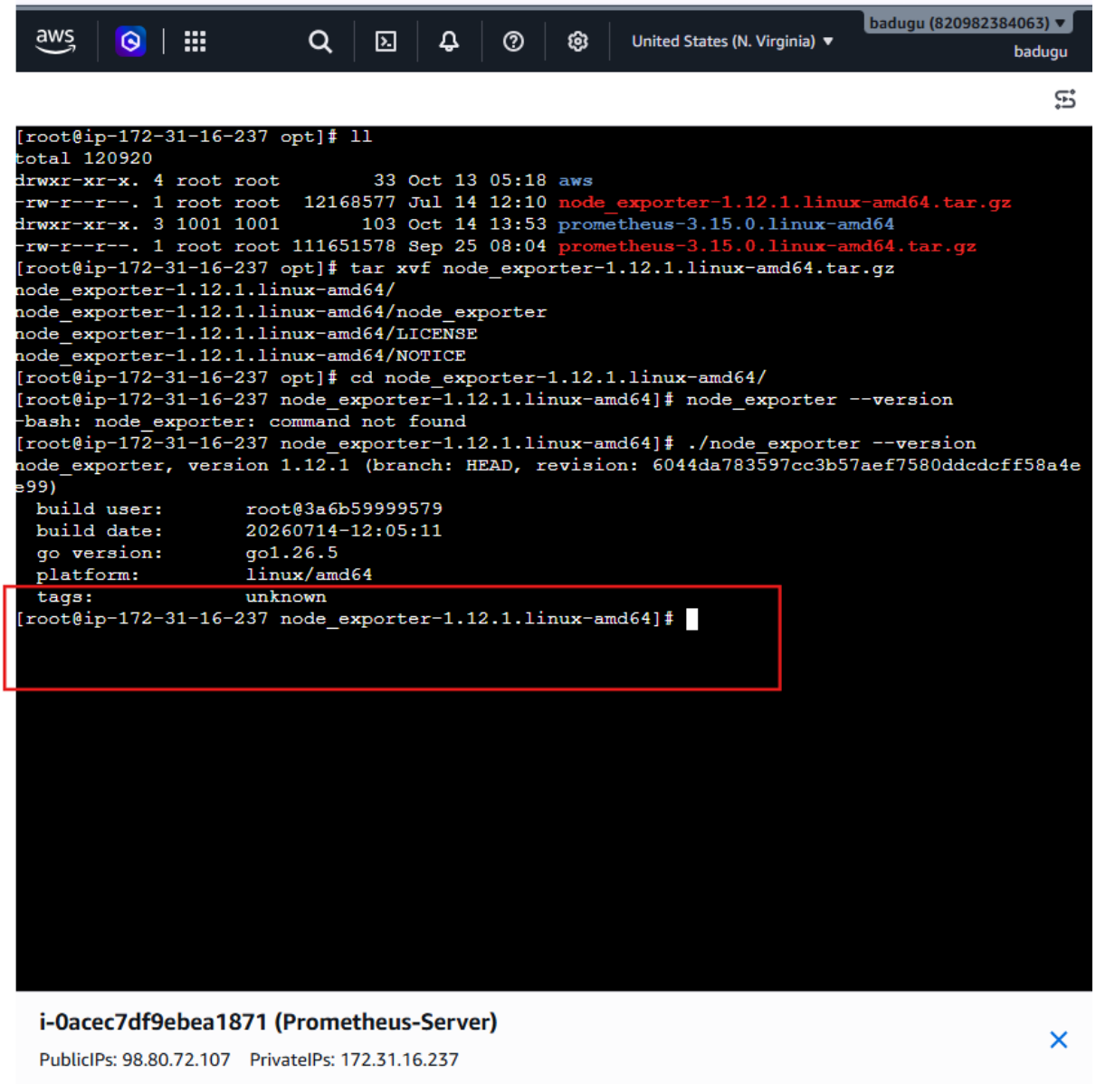
Step:1 Install noder_exporter in Prometheus server only

Sudo cd /opt

Sudo wget https://github.com/prometheus/node_exporter/releases/download/v1.12.1/node_exporter-1.12.1.linux-amd64.tar.gz







Create service file using below data.

nano /etc/systemd/system/node_exporter.service

[Unit]

Description=Node Exporter

After=network.target

[Service]

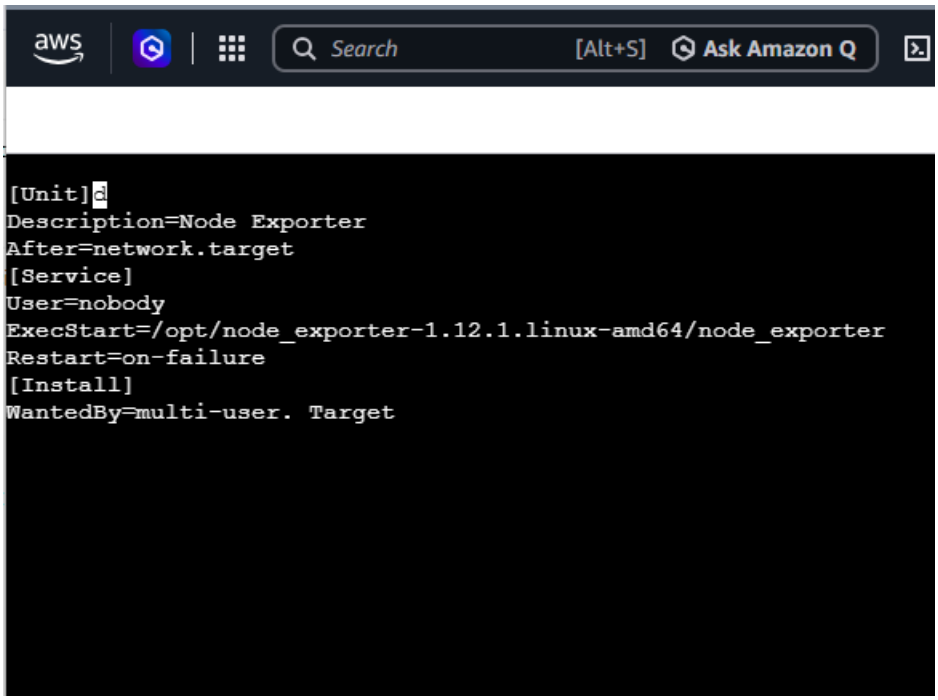
User=nobody

ExecStart=/opt/node_exporter-1.12.1.linux-amd64/node_exporter

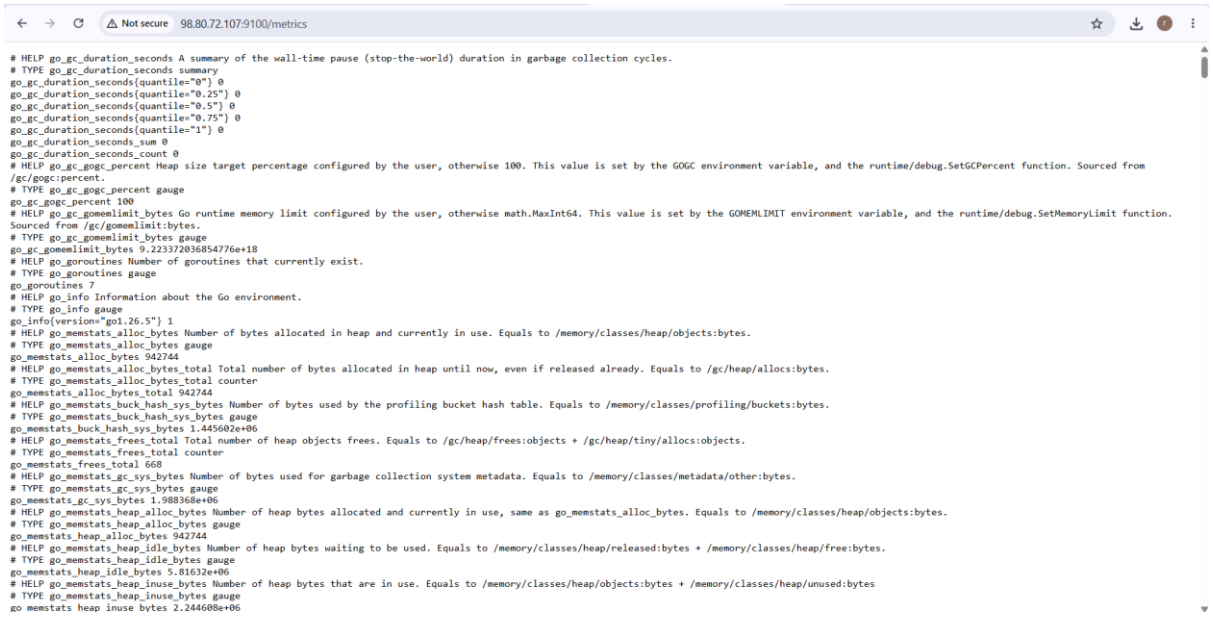
Restart=on-failure

[Install]

WantedBy=multi-user. Target



In Browser: http://98.80.72.107:9100/metrics



Now add node exporter details in promethues.yml file.

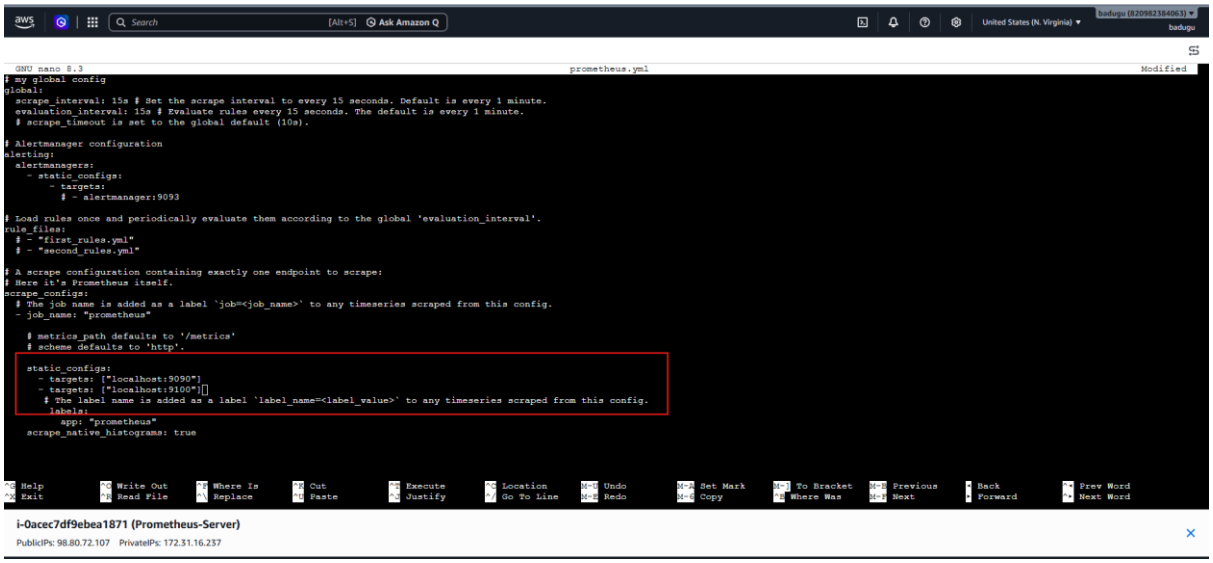
Add the below details

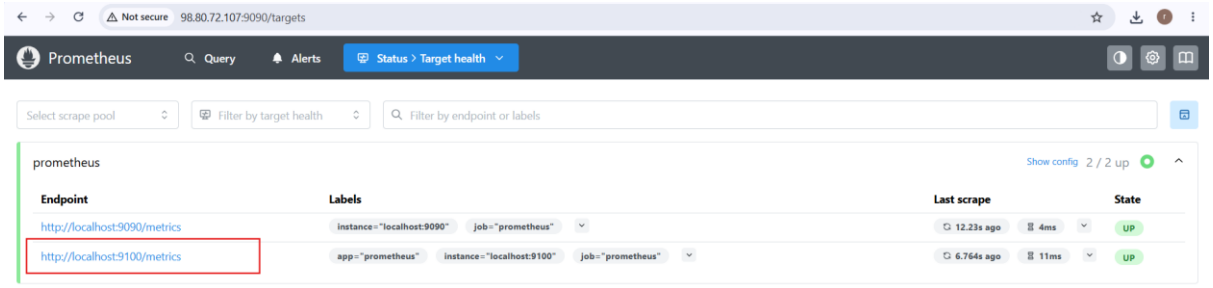
scrape_configs:

- job_name: 'node_exporter'

  static_configs:

    - targets: ['localhost:9100']





You can use below query to check overall cpu from last 1 minute.

100 - (avg by(instance)(irate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)

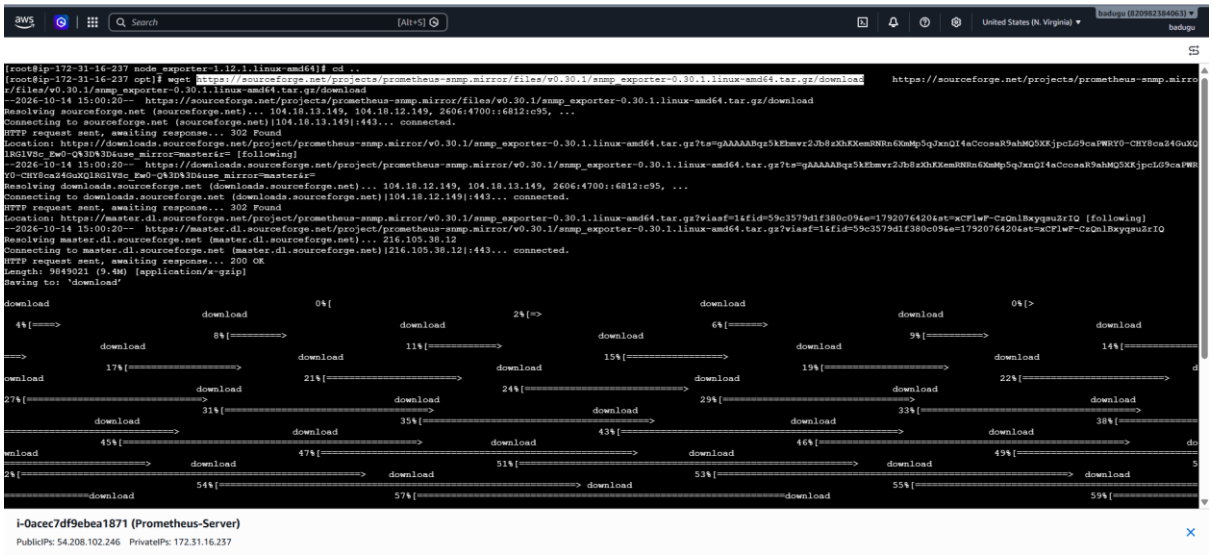


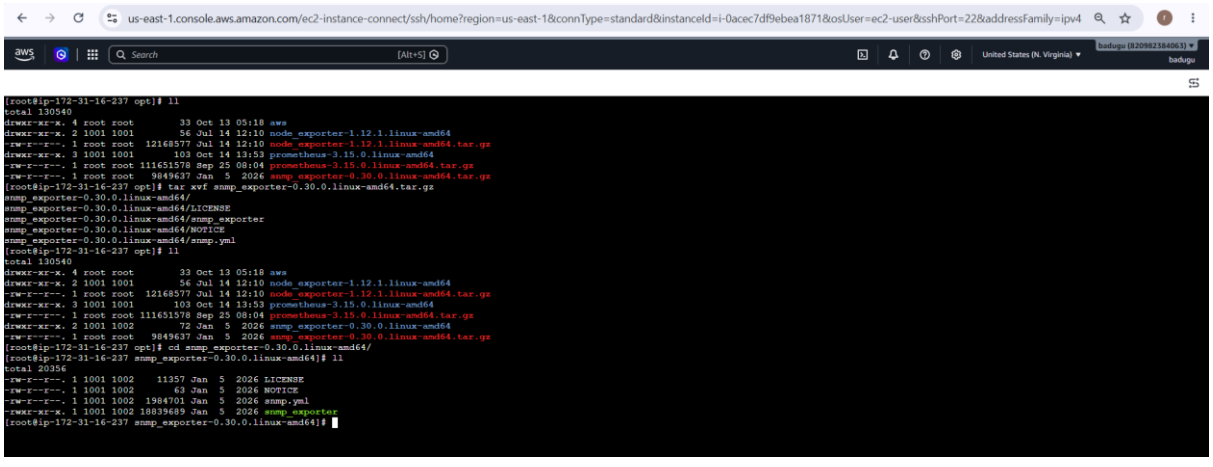
3.Set up SNMP Exporter to monitor network devices.

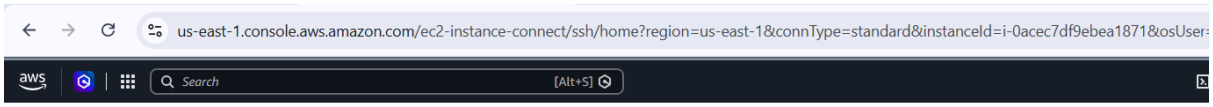
Install snmp_exporter

Cd /opt

Wget https://sourceforge.net/projects/prometheus-snmp.mirror/files/v0.30.1/snmp_exporter-0.30.1.linux-amd64.tar.gz



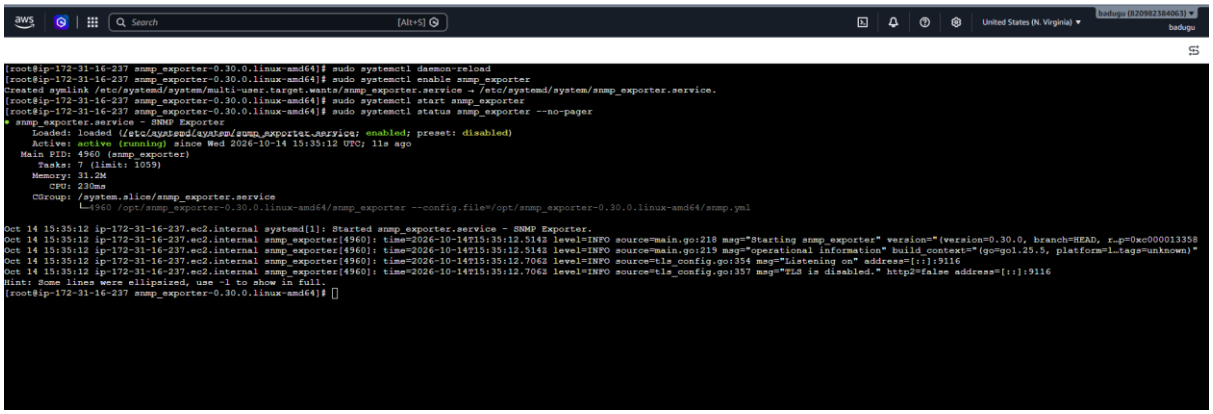


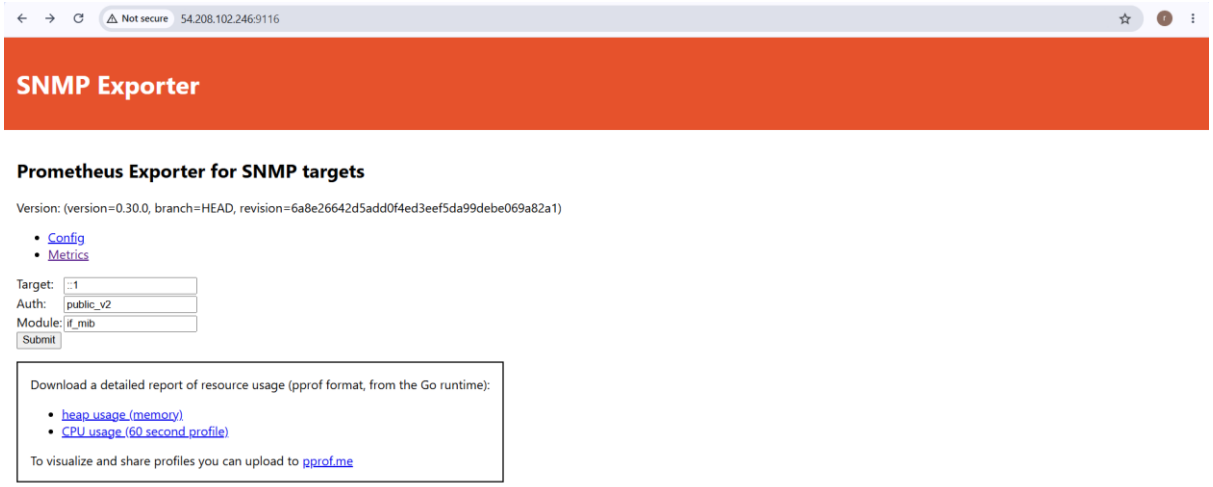


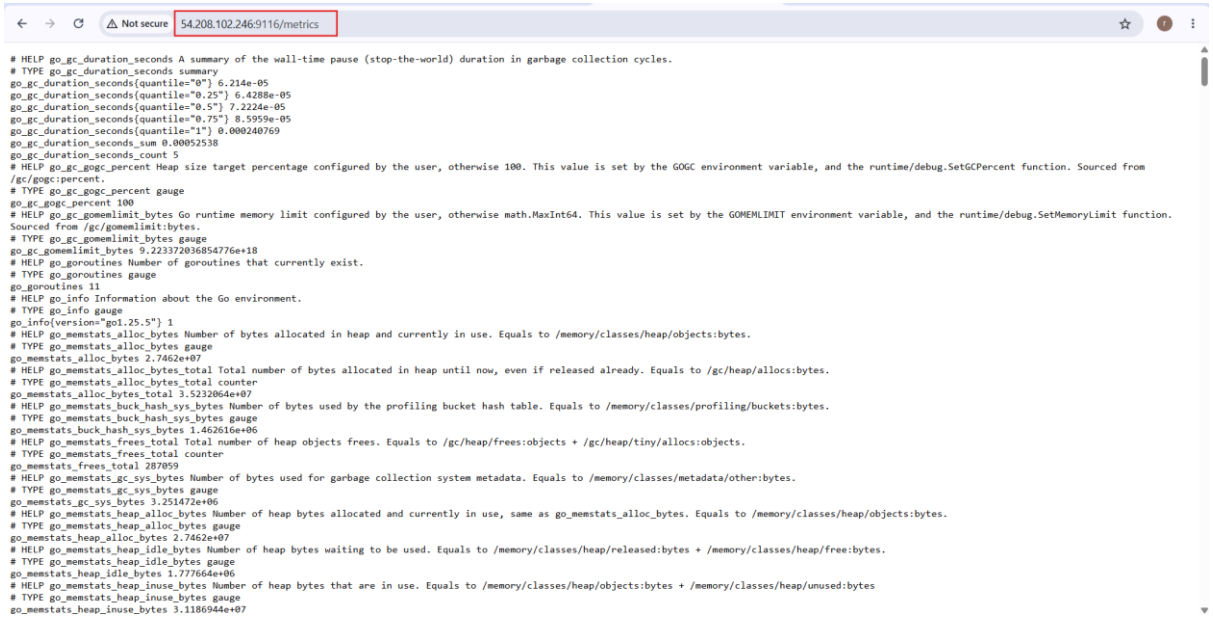

```
[Unit]
Description=SNMP Exporter
Wants=network-online.target
After=network-online.target

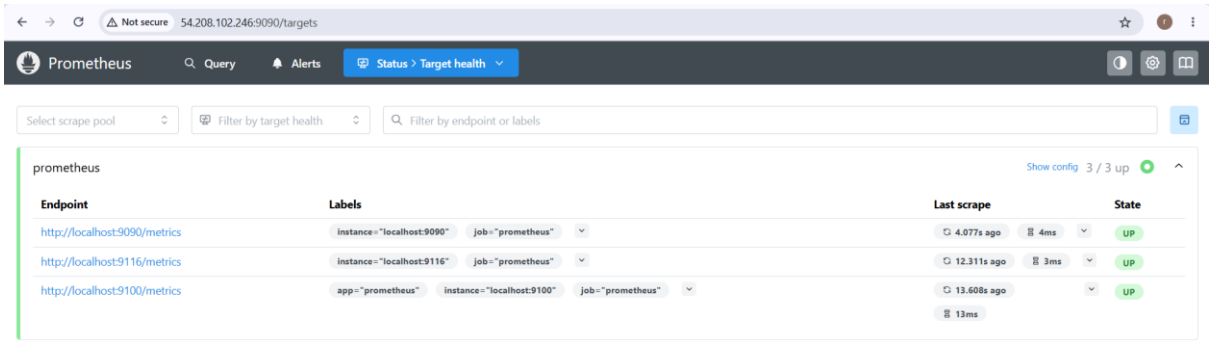
[Service]
Type=simple
User=nobody
ExecStart=/opt/snmp_exporter-0.30.0.linux-amd64/snmp_exporter --config.file=/opt/snmp_exporter-0.30.0.linux-amd64/snmp.yml
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```


[root@ip-172-31-16-237 snmp_exporter-0.30.0.linux-amd64]# sudo systemctl daemon-reload
[root@ip-172-31-16-237 snmp_exporter-0.30.0.linux-amd64]# sudo systemctl enable snmp_exporter
Created symlink /etc/systemd/system/multi-user.target.wants/snmp_exporter.service → /etc/systemd/system/snmp_exporter.service.
[root@ip-172-31-16-237 snmp_exporter-0.30.0.linux-amd64]# sudo systemctl start snmp_exporter
[root@ip-172-31-16-237 snmp_exporter-0.30.0.linux-amd64]# sudo systemctl status snmp_exporter --no-pager
● snmp_exporter.service - SNMP Exporter
Loaded: loaded (/etc/systemd/system/snmp_exporter.service; enabled; preset: disabled)
Active: active (running) since Wed 2026-10-14 15:35:12 UTC; 11s ago
Main PID: 4960 (snmp_exporter)
Tasks: 7 (limit: 1059)
Memory: 31.2M
CPU: 230ms
CGroup: /system.slice/snmp_exporter.service
└─4960 /opt/snmp_exporter-0.30.0.linux-amd64/snmp_exporter --config.file=/opt/snmp_exporter-0.30.0.linux-amd64/snmp.yml
Oct 14 15:35:12 ip-172-31-16-237.ec2.internal systemd[1]: Started snmp_exporter.service - SNMP Exporter.
Oct 14 15:35:12 ip-172-31-16-237.ec2.internal snmp_exporter[4960]: time=2026-10-14T15:35:12.514Z level=INFO source=main.go:218 msg="Starting snmp_exporter" version="(version=0.30.0, branch=HEAD, r
Oct 14 15:35:12 ip-172-31-16-237.ec2.internal snmp_exporter[4960]: time=2026-10-14T15:35:12.514Z level=INFO source=main.go:219 msg="operational information" build_context="(go=go1.25.5, platform=l
Oct 14 15:35:12 ip-172-31-16-237.ec2.internal snmp_exporter[4960]: time=2026-10-14T15:35:12.706Z level=INFO source=tls_config.go:354 msg="Listening on" address=[::]:9116
Oct 14 15:35:12 ip-172-31-16-237.ec2.internal snmp_exporter[4960]: time=2026-10-14T15:35:12.706Z level=INFO source=tls_config.go:357 msg="TLS is disabled." http2=false address=[::]:9116
Hint: Some lines were ellipsized, use -l to show in full.
[root@ip-172-31-16-237 snmp_exporter-0.30.0.linux-amd64]#


Not secure
54.208.102.246:9116
SNMP Exporter
Prometheus Exporter for SNMP targets
Version: (version=0.30.0, branch=HEAD, revision=6a8e26642d5add0f4ed3eef5da99debe069a82a1)
Config
Metrics
Target: ::1
Auth: public_v2
Module: if_mib
Submit
Download a detailed report of resource usage (pprof format, from the Go runtime):
heap usage (memory)
CPU usage (60 second profile)
To visualize and share profiles you can upload to pprof.me


Not secure
54.208.102.246:9116/metrics
HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 6.214e-05
go_gc_duration_seconds{quantile="0.25"} 6.4288e-05
go_gc_duration_seconds{quantile="0.5"} 7.2224e-05
go_gc_duration_seconds{quantile="0.75"} 8.5959e-05
go_gc_duration_seconds{quantile="1"} 0.000240769
go_gc_duration_seconds_sum 0.00052538
go_gc_duration_seconds_count 5
HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and the runtime/debug.SetGCPercent function. Sourced from
/gc/gogc:percent.
TYPE go_gc_gogc_percent gauge
go_gc_gogc_percent 100
HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function.
Sourced from /gc/gomemlimit:bytes.
TYPE go_gc_gomemlimit_bytes gauge
go_gc_gomemlimit_bytes 9.223372036854776e+18
HELP go_goroutines Number of goroutines that currently exist.
TYPE go_goroutines gauge
go_goroutines 11
HELP go_info Information about the Go environment.
TYPE go_info gauge
go_info{version="go1.25.5"} 1
HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.7462e+07
HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 3.5232064e+07
HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.462616e+06
HELP go_memstats_frees_total Total number of heap objects frees. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.
TYPE go_memstats_frees_total counter
go_memstats_frees_total 287059
HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.
TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 3.251472e+06
HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and currently in use, same as go_memstats_alloc_bytes. Equals to /memory/classes/heap/objects:bytes.
TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 2.7462e+07
HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used. Equals to /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.
TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 1.777664e+06
HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes
TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 3.1186944e+07



4.Set up Prometheus to run as a systemd service for automatic startup.

Step 1: Verify the systemd service file

Run:

cat /etc/systemd/system/prometheus.service



Step 2: Reload the systemd configuration

sudo systemctl daemon-reload

This makes systemd reload the service definition.

Step 3: Enable automatic startup

sudo systemctl enable prometheus

Expected output includes a symlink created under multi-user.target.wants.

This ensures Prometheus starts automatically during normal system boot.

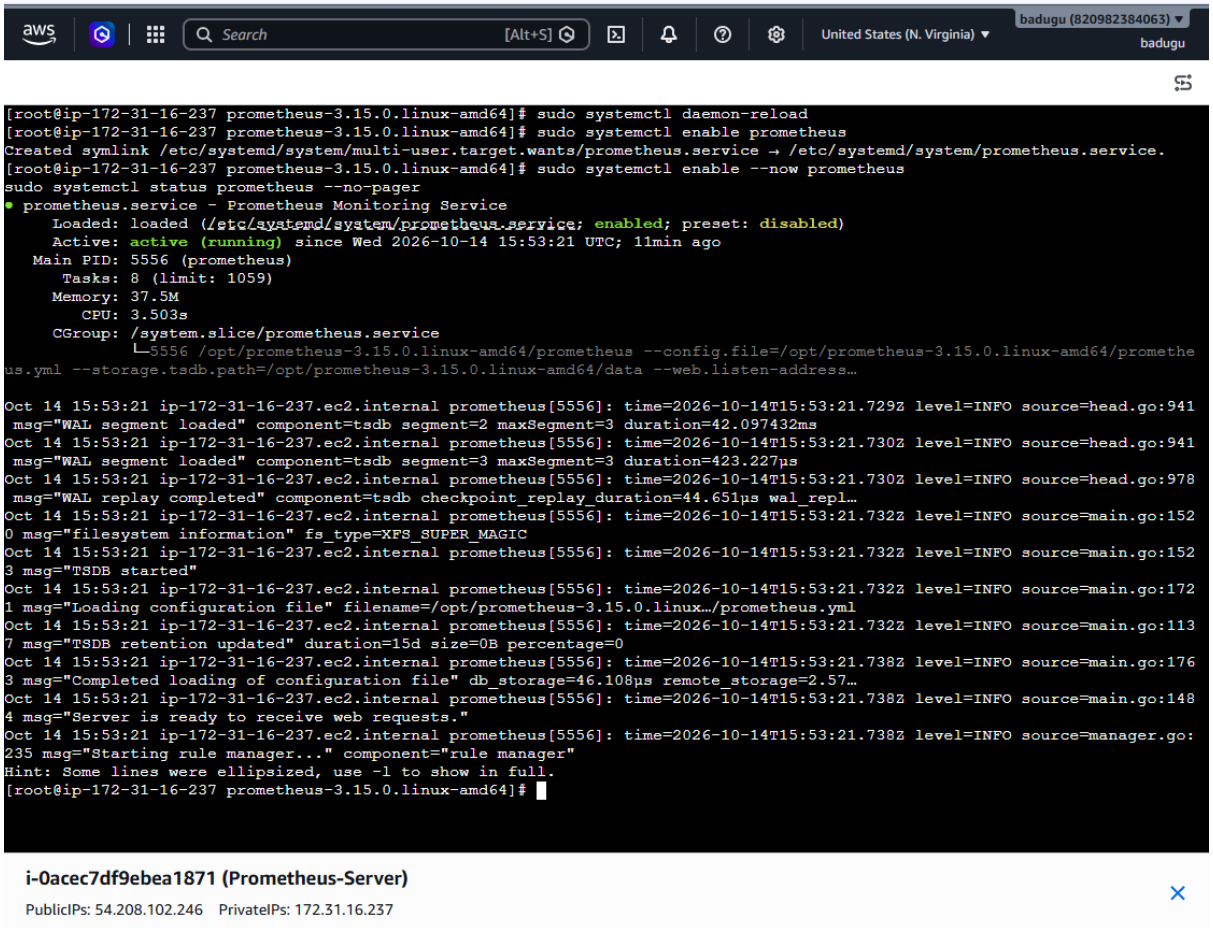
Step 4: Start and verify the service

sudo systemctl enable --now prometheus

sudo systemctl status prometheus --no-pager

Expected status:

Active: active (running)



Check automatic startup is enabled:

sudo systemctl is-enabled prometheus

Expected output:

enabled

Step 5: Test Prometheus health

Run:

curl http://localhost:9090/-/healthy

Expected output:

Prometheus Server is Healthy.

You can also check the service logs:

sudo journalctl -u prometheus -n 20 --no-pager

Final verification: If you want to confirm that Prometheus starts after a reboot, run:

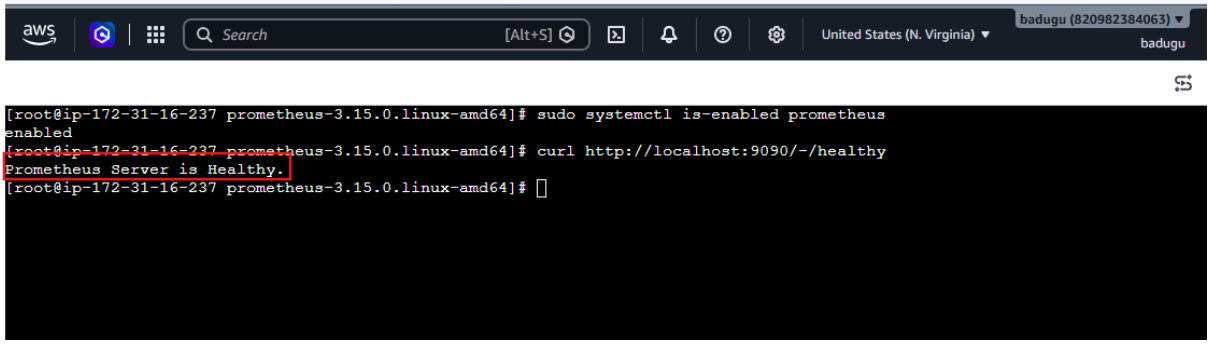
sudo reboot

Reconnect to your EC2 instance after it comes back online, then run:

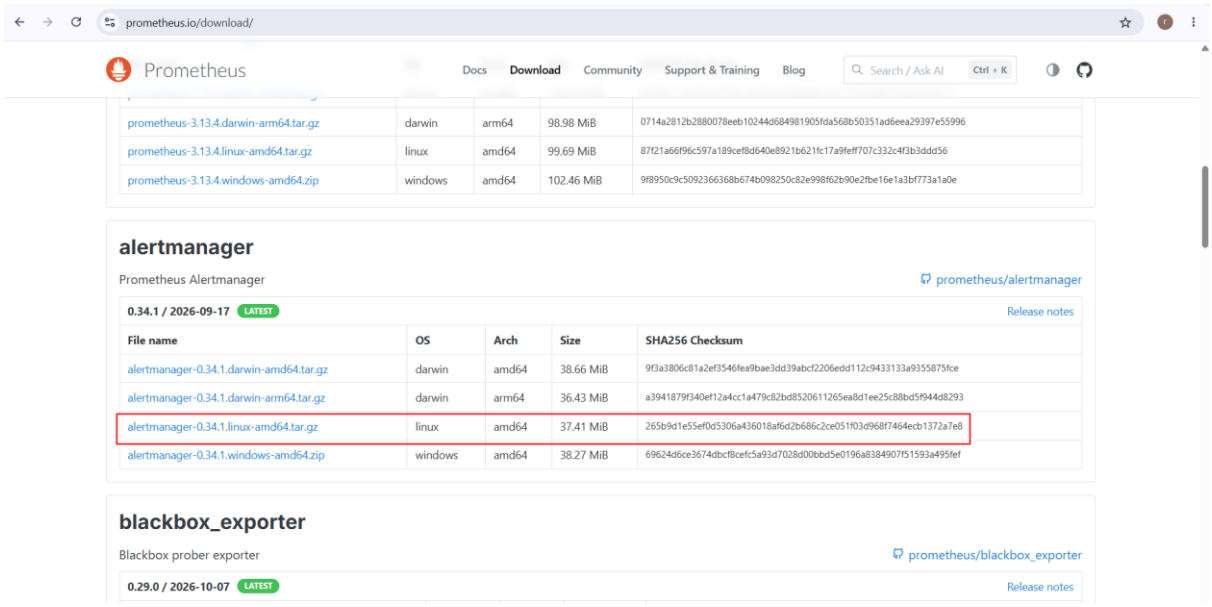
sudo systemctl status prometheus --no-pager

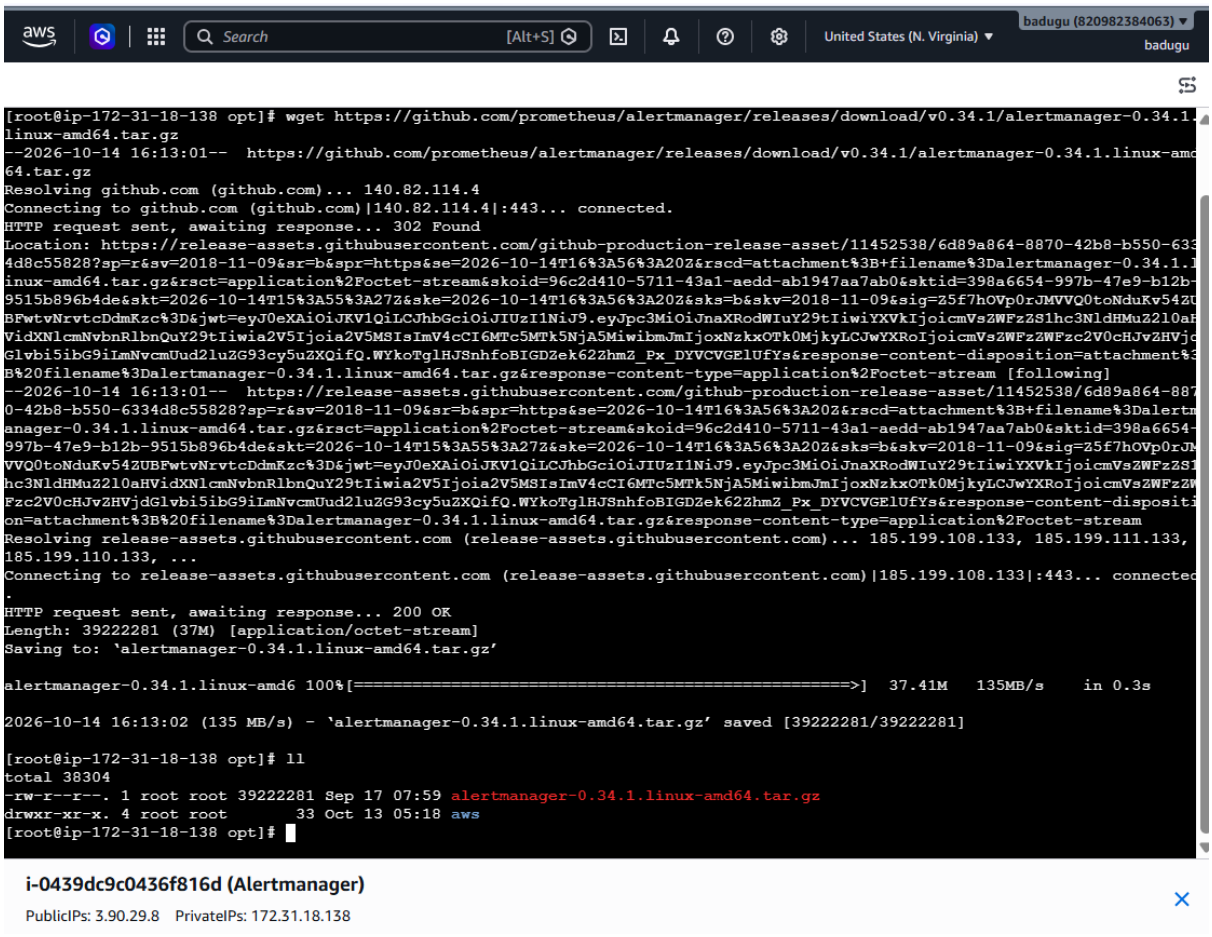
Confirm it shows active (running).

Result: Prometheus is configured to start automatically at boot, restart after service failures, and run under the root user.



5.Install Alertmanager and configure an email alert.





Step 1: Understand the architecture

Prometheus

Monitors targets and evaluates alert rules

Alertmanager

Receives alerts and routes notifications

Gmail

Sends the email alert to your inbox

Step 2: Create the Alertmanager configuration directory

You are logged in as root on your server. Run:

mkdir -p /etc/alertmanager

mkdir -p /var/lib/alertmanager

Copy the existing configuration file:

cp alertmanager.yml /etc/alertmanager/alertmanager.yml

Confirm the file exists:

ls -l /etc/alertmanager/

If the copy command says the file cannot be found, use the full path to your extracted directory.

Step 3: Generate a Gmail App Password

Alertmanager needs SMTP credentials to send email. For Gmail, use an App Password, not your normal Gmail account password.

1. Sign in to the Google account from which you want to send alerts.
2. Open Google Account Security.
3. Enable 2-Step Verification, if it is not already enabled.
4. Open Google App Passwords.
5. Create an app password, for example, named Alertmanager.
6. Copy the generated password.

If Google does not show the App Password option, your account type or security policy may not support it.

Important: Keep the app password secret. Do not paste it into this chat or commit it to Git.

Step 4: Configure Alertmanager for Gmail

Open the configuration file:

vi /etc/alertmanager/alertmanager.yml

Replace its contents with the following configuration. Change the sender email, recipient email, and app password to your own values.

```
global:
  resolve_timeout: 5m
  smtp_smarthost: 'smtp.gmail.com:587'
  smtp_from: 'sender@gmail.com'
  smtp_auth_username: 'sender@gmail.com'
  smtp_auth_password: 'YOUR_GMAIL_APP_PASSWORD'
  smtp_require_tls: true

route:
  receiver: 'gmail-notifications'
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 1h

receivers:
  - name: 'gmail-notifications'
    email_configs:
      - to: 'recipient@gmail.com'
        send_resolved: true
```

Replace these values:

- sender@gmail.com: your Gmail account used to send emails.
- recipient@gmail.com: the inbox where you want to receive alerts.
- YOUR_GMAIL_APP_PASSWORD: the Google App Password you generated in Step 3.

You can send alerts to the same Gmail address or to a different one.

Save the file in vi by pressing Esc, typing :wq, and pressing Enter.

Secure the configuration file

Because it contains credentials, restrict access:

chown root:root /etc/alertmanager/alertmanager.yml

chmod 600 /etc/alertmanager/alertmanager.yml

This simple configuration stores the app password in the YAML file. For a production environment, prefer a protected secret file or secrets-management solution supported by your deployment.

Step 5: Validate the configuration

Copy the Alertmanager binaries into a standard system directory:

install -m 0755 alertmanager /usr/local/bin/alertmanager

install -m 0755 amtool /usr/local/bin/amtool

Check your YAML configuration:

/usr/local/bin/amtool check-config /etc/alertmanager/alertmanager.yml

If it succeeds, you should see a message indicating that the configuration is valid.

If you receive a YAML or configuration error, fix that before starting the service.

Step 6: Create a systemd service

This lets Alertmanager start automatically and restart if it fails.

Create a dedicated service account and ensure the data directory is owned by it:

useradd --system --no-create-home --shell /sbin/nologin alertmanager

chown -R alertmanager:alertmanager /var/lib/alertmanager

Create the service file:

vi /etc/systemd/system/alertmanager.service

Paste this configuration:

```
[Unit]
Description=Prometheus Alertmanager
Wants=network-online.target
After=network-online.target

[Service]
User=alertmanager
Group=alertmanager
Type=simple
ExecStart=/usr/local/bin/alertmanager \
  --config.file=/etc/alertmanager/alertmanager.yml \
  --storage.path=/var/lib/alertmanager \
  --web.listen-address=0.0.0.0:9093
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Save the file and run:

```
systemctl daemon-reload
systemctl enable --now alertmanager
```

Check the status:

```
systemctl status alertmanager --no-pager
```

You want to see:

Active: active (running)

If it fails, check the logs:

journalctl -u alertmanager -n 50 --no-pager

Step 7: Open the Alertmanager web interface

Check whether Alertmanager is listening on port 9093:

ss -lntp | grep 9093

Get your EC2 private and public IP addresses:

hostname -I

curl -s https://checkip.amazonaws.com

Open this URL in your browser:

http://YOUR_EC2_PUBLIC_IP:9093

If the page does not open, check the EC2 security group's inbound rules. For learning purposes, you may allow TCP port 9093 from your own public IP address only. Avoid exposing it to the entire internet.

The Alertmanager web page should display its status and alert information.

Step 8: Connect Prometheus to Alertmanager

Alertmanager cannot send alerts from Prometheus until Prometheus is configured to forward them.

First, check whether Prometheus is running on the same EC2 instance:

systemctl status prometheus --no-pager

If it is installed on the same server, open:

vi /etc/prometheus/prometheus.yml

Add or merge the following sections into the existing configuration. Do not delete your existing scrape_configs section.

alerting:

  alertmanagers:

```
  - static_configs:
      - targets:
          - 'localhost:9093'


rule_files:
  - '/etc/prometheus/alert_rules.yml'
```

If Prometheus runs on another server, replace localhost:9093 with the Alertmanager server's private IP and port, and permit the required network traffic between the servers.

Step 9: Create a test alert rule

Create the rules file:

vi /etc/prometheus/alert_rules.yml

Add this test rule:

```
groups:
  - name: email-alert-test
    rules:
      - alert: TestEmailAlert
        expr: vector(1)
        for: 10s
        labels:
          severity: warning
        annotations:
          summary: "Test email alert from Prometheus"
          description: "This is a test alert to verify the Prometheus and Alertmanager email configuration."
```

This test expression always returns 1, so the alert should fire after approximately 10 seconds. It is only for testing; remove it when testing is complete.

Validate the rule file:

promtool check rules /etc/prometheus/alert_rules.yml

If promtool is not in your PATH, locate it:

which promtool

If Prometheus is installed on a separate machine, create and validate the rule file on that machine instead.

Step 10: Restart Prometheus

After validating the rule and updating prometheus.yml, run:

systemctl restart prometheus

systemctl status prometheus --no-pager

Check the logs if it fails:

journalctl -u prometheus -n 50 --no-pager

Now check the Prometheus web interface, normally at:

http://YOUR_PROMETHEUS_SERVER_IP:9090/alerts

You should see the TestEmailAlert rule. After its pending period, it should transition to Firing.

Step 11: Verify the email in Gmail

Open the Alertmanager interface:

http://YOUR_ALERTMANAGER_SERVER_IP:9093

Look for TestEmailAlert under the active alerts.

Within a short time, check the recipient's Gmail inbox, including the Spam folder. You should receive an email with the subject or content related to TestEmailAlert.

You can also inspect Alertmanager's logs:

journalctl -u alertmanager -n 100 --no-pager

If the email is not received, check the following:

- The sender and recipient email addresses are correct.
- The Gmail App Password is correct and belongs to the sender account.
- Gmail SMTP port 587 is reachable from the EC2 server.
- The Alertmanager configuration passed validation.
- The Prometheus alert is actually firing.
- The EC2 network and security settings allow the required outbound SMTP traffic.

Step 12: Remove the test alert

After confirming email delivery, remove the TestEmailAlert rule from /etc/prometheus/alert_rules.yml and validate the file again. Then restart Prometheus:

promtool check rules /etc/prometheus/alert_rules.yml

systemctl restart prometheus

Replace the test rule with meaningful production alerts, such as an unavailable target, high CPU usage, high memory usage, or low disk space.

Final verification checklist

Setup progress

0 of 7 complete

Alertmanager configuration is valid

Alertmanager service is active and enabled

Alertmanager web interface opens

Prometheus is connected to Alertmanager

TestEmailAlert reaches the Firing state

Test email arrives in the recipient inbox

Test rule is removed after successful verification

Important: Your next step depends on where Prometheus is installed. If you share the output of:

systemctl status prometheus --no-pager

systemctl status alertmanager --no-pager

I can guide you through the exact next commands for your server.

6.Install Grafana and add Prometheus as a data source.

Step-by-Step Guide: Install Grafana and Add Prometheus as a Data Source

In this lab, you will install Grafana on your Linux EC2 instance, connect it to Prometheus, and create a dashboard to visualize monitoring metrics.

Since you have already been working with Prometheus and exporters on your AWS EC2 instances, we will use that setup for this procedure.

1. Understand the architecture

Linux EC2 / Exporters

Node Exporter · SNMP Exporter

Prometheus

Collects and stores metrics

Default port: 9090

Grafana

Queries Prometheus and displays dashboards

Default port: 3000

Important: Grafana does not collect metrics directly from your exporters.
Prometheus collects the metrics, and Grafana queries Prometheus to display them.

2. Prerequisites

Before starting, make sure you have:

- An AWS EC2 instance running Linux, such as Amazon Linux 2023.
- Prometheus installed and running.
- Access to the EC2 instance using SSH or a terminal.
- An EC2 security group that you can update.

Run these commands on the EC2 instance where you plan to install Grafana.

3. Check whether Prometheus is running

Run:

sudo systemctl status prometheus --no-pager

If you installed Prometheus manually rather than as a systemd service, check the process and listening port:

ps -ef | grep '[p]rometheus'

sudo ss -lntp | grep ':9090'

Open Prometheus locally on the EC2 instance:

curl -s http://localhost:9090/-/ready

Expected output:

Prometheus Server is Ready.

If Prometheus is on a different EC2 instance, you must use that instance's reachable private IP address later instead of localhost.

4. Install Grafana on Amazon Linux 2023 / RHEL

The following commands use Grafana's official RPM repository.



Grafana documentation

Step 4.1 — Import the Grafana signing key

sudo dnf install -y wget

wget -q -O /tmp/grafana-gpg.key https://rpm.grafana.com/gpg.key

sudo rpm --import /tmp/grafana-gpg.key

Step 4.2 — Configure the repository

Copy and run this complete command:

sudo tee /etc/yum.repos.d/grafana.repo <<'EOF'

[grafana]

name=grafana

baseurl=https://rpm.grafana.com

repo_gpgcheck=1

enabled=1

gpgcheck=1

gpgkey=https://rpm.grafana.com/gpg.key

sslverify=1

sslcacert=/etc/pki/tls/certs/ca-bundle.crt

EOF

Step 4.3 — Install Grafana

sudo dnf install -y grafana

Wait for the installation to finish.

Step 4.4 — Start and enable Grafana

sudo systemctl daemon-reload

sudo systemctl enable --now grafana-server

Step 4.5 — Verify the service

sudo systemctl status grafana-server --no-pager

You should see:

Active: active (running)

Check whether port 3000 is listening:

sudo ss -lntp | grep ':3000'

5. Configure the AWS security group

Grafana uses TCP port 3000 by default.

1. Open the AWS Management Console.
2. Navigate to EC2 → Instances.
3. Select the instance where Grafana is installed.
4. Open its Security tab.
5. Click the associated security group.

6. Select Edit inbound rules → Add rule.

Use these settings:

Setting	Value
Type	Custom TCP
Port range	3000
Source	Your public IP address /32
Description	Grafana Web UI

Save the rules.

Security recommendation: Do not expose Grafana to 0.0.0.0/0 for this lab unless you have a specific, protected access plan. If Prometheus is on another EC2 instance, allow port 9090 from the Grafana instance's security group or private network as appropriate. Avoid exposing Prometheus publicly.

6. Open Grafana in your browser

Find the public IPv4 address of the EC2 instance running Grafana.

Open this URL in your browser:

http://YOUR-GRAFANA-EC2-PUBLIC-IP:3000

For example, if the instance's public IP were 203.0.113.10, you would use:

http://203.0.113.10:3000

The IP above is an example, not your actual server address.

Log in using the initial credentials:

Field	Value
Username	admin
Password	admin

Grafana will normally ask you to change the default password at first login. Set a strong password and keep it safe.

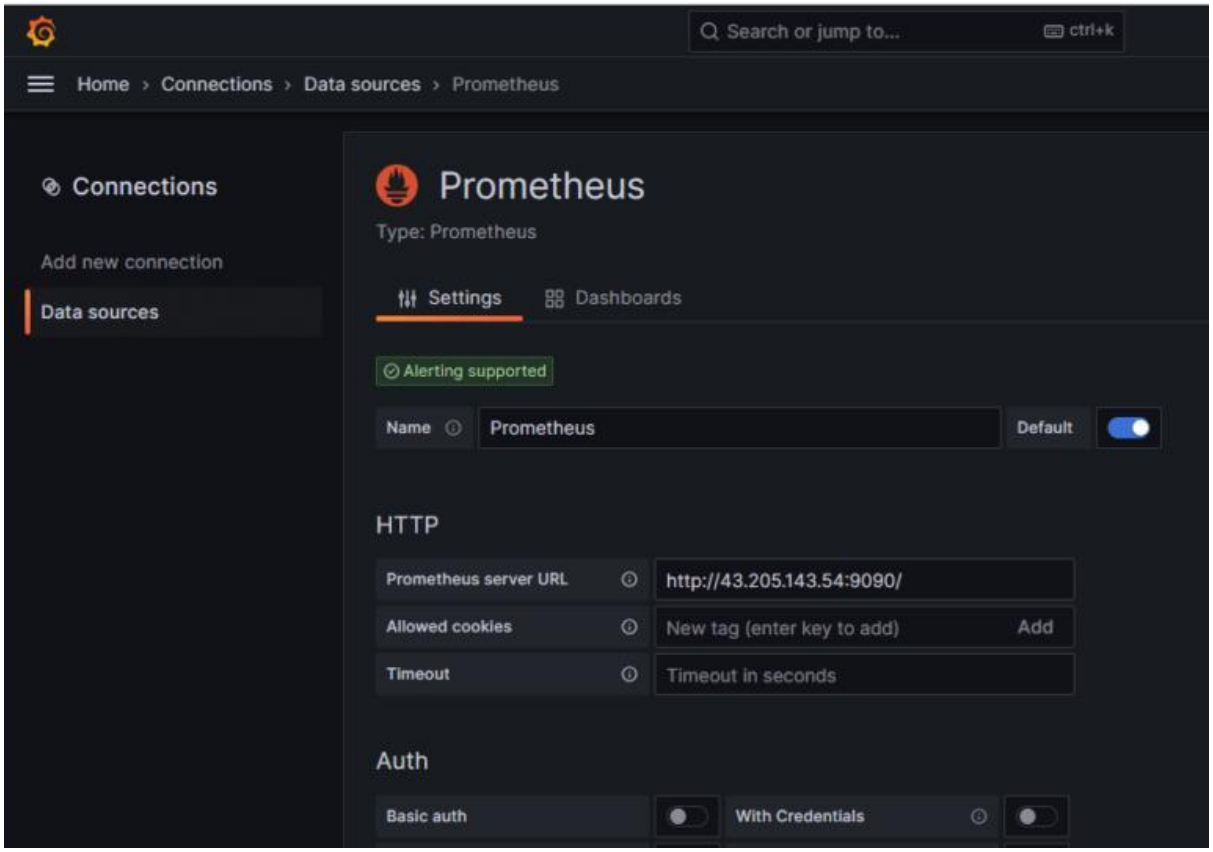


Prometheus

7. Add Prometheus as a data source

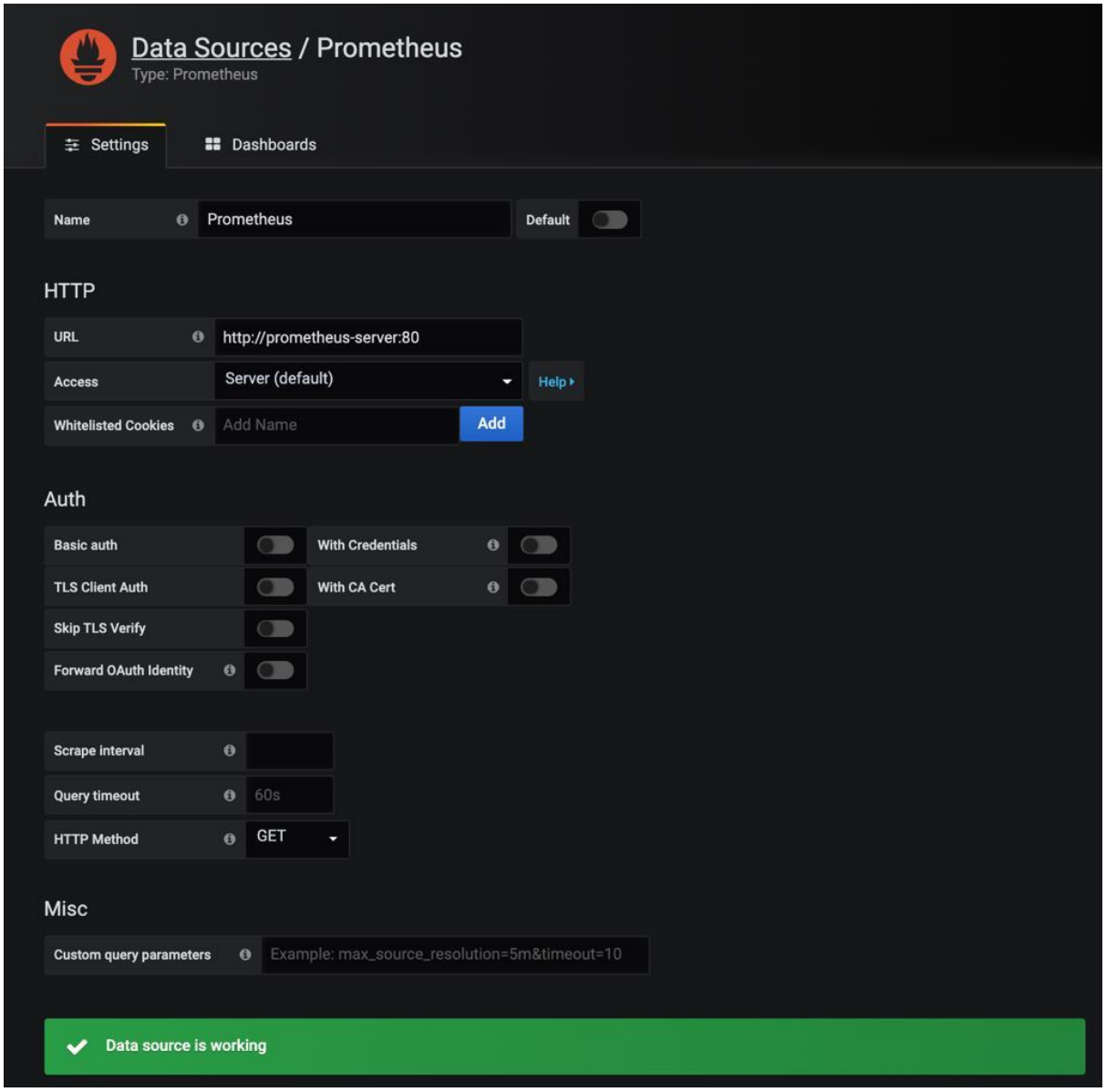
Grafana includes a built-in Prometheus data source, so you do not need to install a separate plugin.

Grafana documentation



Step 7.1 — Open Connections

In Grafana, use the left navigation menu and select Connections → Data sources or Connections → Add new connection, depending on your version.



Step 7.2 — Select Prometheus

Search for Prometheus, select the Prometheus data source, and click Add new data source if prompted.

Step 7.3 — Configure the URL

Use the appropriate URL for your architecture:

Case A: Prometheus and Grafana are on the same EC2 instance

http://localhost:9090

Case B: Prometheus and Grafana are on different EC2 instances

http://PROMETHEUS-PRIVATE-IP:9090

Replace PROMETHEUS-PRIVATE-IP with the actual private IP address of the Prometheus server.

For example, if Prometheus is running at private IP 172.31.18.138, enter:

http://172.31.18.138:9090

Use that address only if it is actually the private IP of your Prometheus instance and your VPC/security-group rules permit the connection.

Step 7.4 — Save and test

1. Keep the default server-side access setting.
2. Scroll down to the bottom of the configuration page.
3. Click Save & test.

Expected success message:

Successfully queried the Prometheus API.

This confirms that Grafana can communicate with Prometheus.



Grafana documentation

8. Verify that Grafana can retrieve metrics

Now test the data source before building a dashboard.

1. In Grafana, open Explore from the left navigation menu.
2. Select your Prometheus data source.
3. Enter this PromQL query:

up

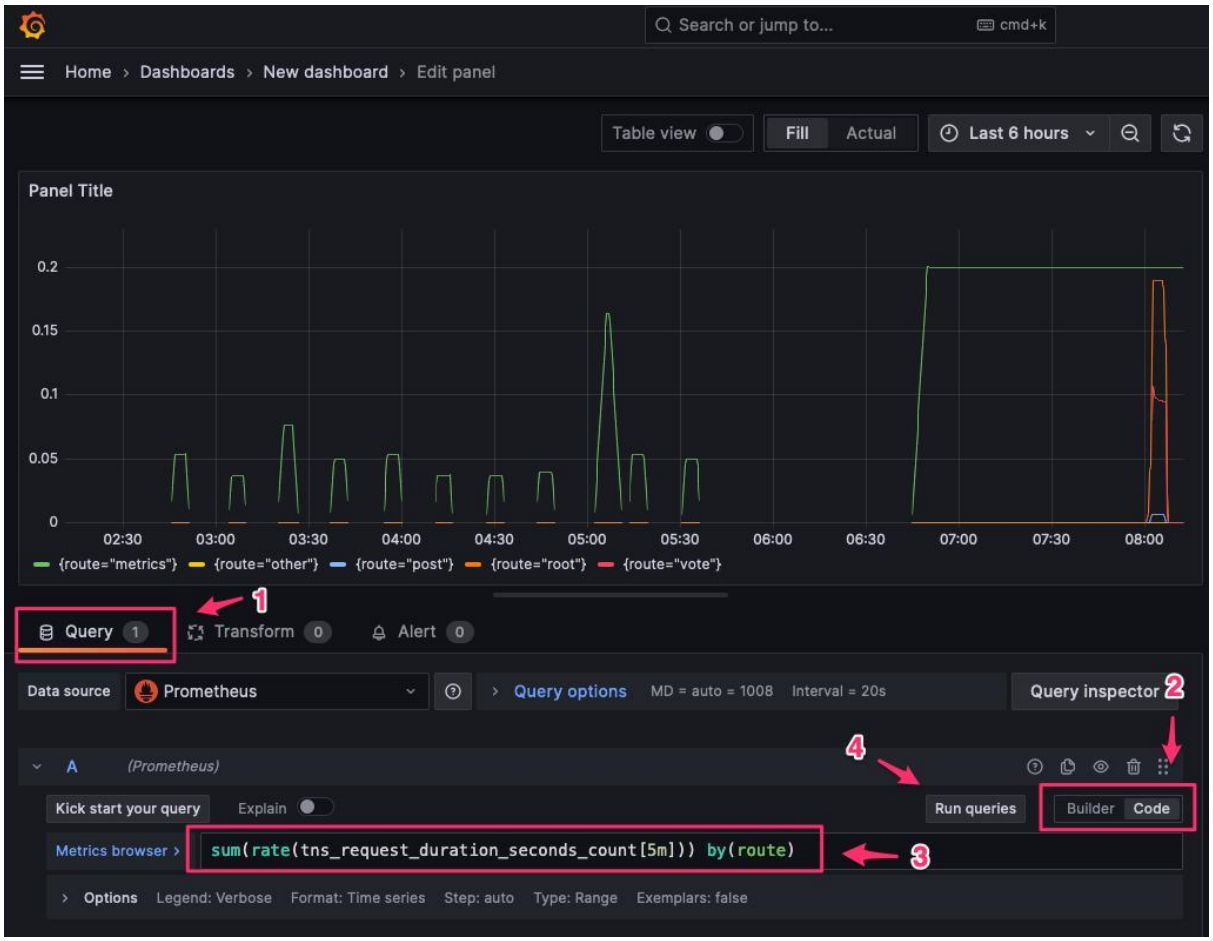
4. Click Run query.

Expected result: one or more time series with values of 1 or 0.

- 1 means Prometheus successfully scraped that target.
- 0 means Prometheus knows about the target but the scrape failed.

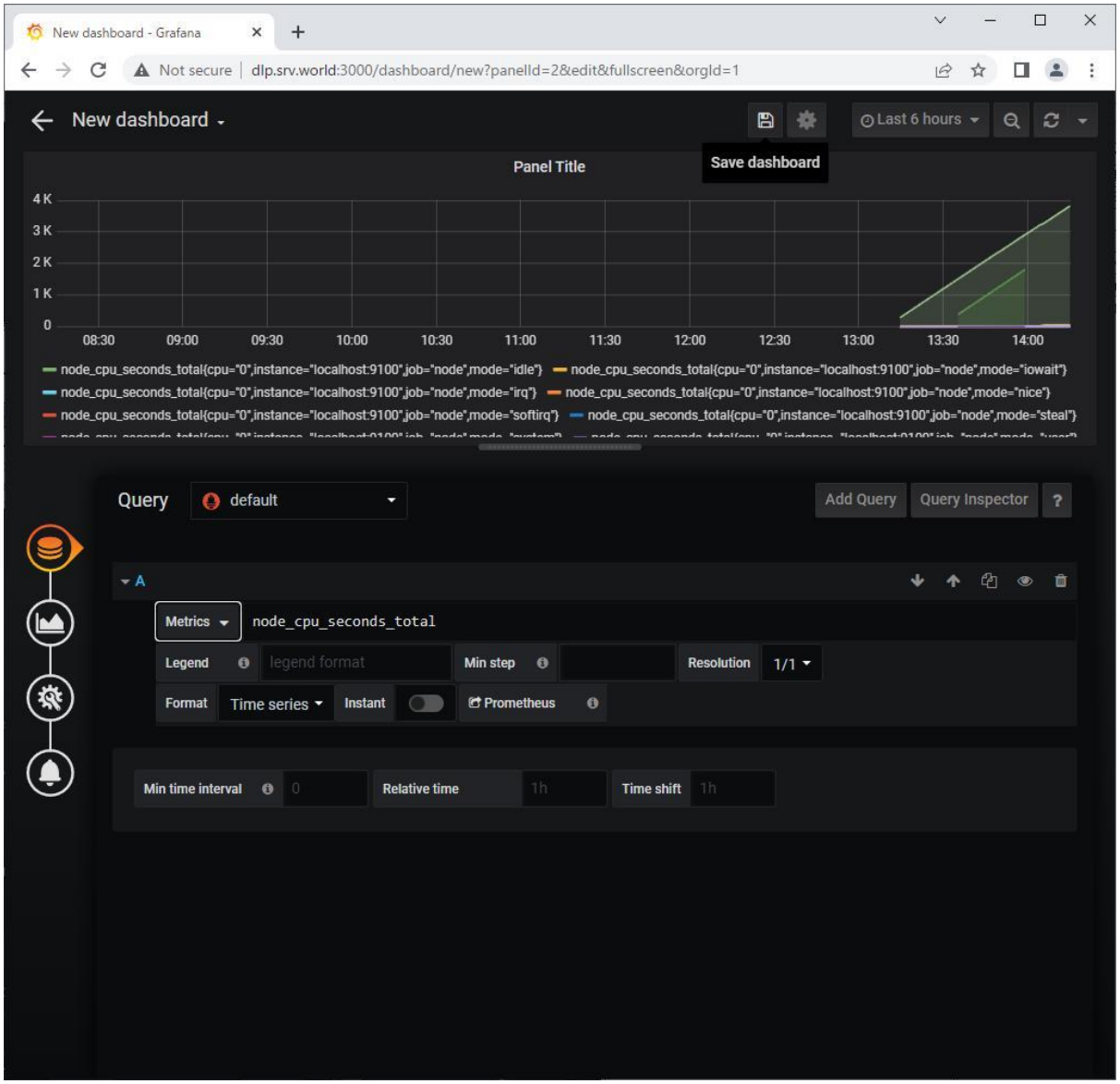
If the query returns no data, Prometheus may not yet have any targets producing metrics. Check Status → Targets in the Prometheus UI.

9. Create your first Grafana dashboard



Step 9.1 — Create a dashboard

Navigate to Dashboards → New → New dashboard, then click Add visualization.



Step 9.2 — Add a query

Select the Prometheus data source and enter one of the queries below. Choose a suitable visualization such as Time series or Stat.

Try these example queries:

What to monitor	**PromQL query**
Target health	up
CPU usage by host, percentage	100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))

Available memory, bytes	node_memory_MemAvailable_bytes
Total memory, bytes	node_memory_MemTotal_bytes
Disk available, bytes	node_filesystem_avail_bytes{mountpoint="/"}

The CPU, memory, and disk queries require Node Exporter metrics to be collected by Prometheus. If your configuration uses different labels or filesystem mount points, adjust the queries accordingly.

Step 9.3 — Save the dashboard

1. Set the dashboard name to EC2 System Monitoring.
2. Click Save dashboard.
3. Open the dashboard and confirm that the panels display data.

You now have a basic monitoring dashboard using Grafana and Prometheus.

10. Troubleshooting common errors

Problem	**What to check**
Grafana page does not open	Run sudo systemctl status grafana-server and verify inbound TCP 3000.
Connection refused on port 9090	Check that Prometheus is running and listening on the expected address.
Data source test fails	Verify the URL, network connectivity, and EC2 security-group rules.
up returns no results	Open Prometheus → Status → Targets and verify scrape configuration.
CPU or memory queries show no data	Confirm that Node Exporter is running and its target is UP.

If Grafana cannot connect to Prometheus, test connectivity from the Grafana EC2 instance:

curl -v http://localhost:9090/-/ready

Use the Prometheus private IP instead of localhost if the services run on different instances. Grafana's troubleshooting documentation covers these connection checks.



Grafana documentation

Final verification checklist

Grafana lab completion

0 of 7

Grafana package installed

grafana-server service is active

Grafana UI opens on port 3000

Logged in and changed the default password

Prometheus data source saved and tested successfully

Prometheus up query returns data

Dashboard created and saved

Official references for your lab:

- Grafana installation on Amazon Linux / RHEL-based systems
- Configure Prometheus as a Grafana data source
- Prometheus and Grafana visualization guide

7.Configure Prometheus for service discovery of dynamic targets.

Configure Prometheus for Service Discovery of Dynamic Targets

In this lab, you will configure Prometheus to automatically discover monitoring targets instead of manually adding every server or exporter to prometheus.yml.

Since you are setting up Prometheus, Alertmanager, exporters, and Grafana on EC2, we will use Prometheus file-based service discovery as the beginner-friendly example. You can later extend this to AWS EC2 service discovery.

1. What is service discovery?

Normally, you might configure a target manually in prometheus.yml:

```
scrape_configs:
  - job_name: 'node-exporter'
    static_configs:
      - targets:
          - '172.31.16.237:9100'
          - '172.31.18.138:9100'
```

The problem is that whenever a server is added, removed, or its IP address changes, you may need to edit the configuration.

With dynamic service discovery, Prometheus obtains targets from a discovery source and updates its target list automatically.

Target inventory

JSON file containing exporter addresses

Prometheus service discovery

Reads target definitions and refreshes them

Node Exporters

Prometheus scrapes metrics on port 9100

There are several discovery methods:

- File-based discovery: reads targets from JSON or YAML files.
- AWS EC2 discovery: discovers instances using AWS APIs and filters.
- Kubernetes discovery: discovers pods, services, and endpoints.
- DNS discovery: discovers targets through DNS records.

We will first implement file-based discovery because it is easy to test and understand.

2. Check your Prometheus installation

Run these commands on your Prometheus EC2 instance.

sudo systemctl status prometheus --no-pager

If you installed Prometheus manually and do not have a systemd service, check the process:

ps -ef | grep '[p]rometheus'

Check whether Prometheus is responding:

curl -s http://localhost:9090/-/ready

Expected output:

Prometheus Server is Ready.

3. Find the Prometheus configuration file

If Prometheus is installed manually, your configuration might be in the extracted Prometheus directory. If installed as a service, it is commonly /etc/prometheus/prometheus.yml.

Check the actual service configuration:

sudo systemctl cat prometheus

Look for the --config.file argument. For example:

--config.file=/etc/prometheus/prometheus.yml

Set the path below to match your installation:

export PROM_CONFIG=/etc/prometheus/prometheus.yml

sudo cat "$PROM_CONFIG"

Important: Do not replace your entire existing configuration. We will add a new scrape job while preserving your existing jobs for Prometheus, SNMP Exporter, and other targets.

4. Create a directory for dynamic targets

Create a directory for the target inventory:

sudo mkdir -p /etc/prometheus/file_sd

Create a JSON file:

sudo nano /etc/prometheus/file_sd/node_targets.json

Add the following content. Replace the example IP addresses with the private IP addresses of your actual EC2 instances running Node Exporter.

```
[
  {
    "targets": [
      "172.31.16.237:9100"
    ],
    "labels": {
      "environment": "dev",
      "team": "monitoring"
    }
  },
  {
    "targets": [
      "172.31.18.138:9100"
    ],
    "labels": {
      "environment": "prod",
      "team": "monitoring"
    }
  }
]
```

Save the file:

- Press Esc.
- Type :wq.
- Press Enter.

The IP addresses above are examples; use your real target addresses.

Set permissions so Prometheus can read the file:

sudo chmod 755 /etc/prometheus/file_sd

sudo chmod 644 /etc/prometheus/file_sd/node_targets.json

Validate the JSON:

python3 -m json.tool /etc/prometheus/file_sd/node_targets.json

File-based discovery supports JSON and YAML target definitions and watches the configured files for changes.



Prometheus

+1

5. Configure file-based service discovery

Open the Prometheus configuration:

sudo nano /etc/prometheus/prometheus.yml

Under the existing scrape_configs: section, add this job:

```
  - job_name: 'dynamic-node-exporters'
    scrape_interval: 15s
    file_sd_configs:
      - files:
          - '/etc/prometheus/file_sd/node_targets.json'
        refresh_interval: 30s
```

Your configuration should have this general structure:

```
global:
```

```
  scrape_interval: 15s
  evaluation_interval: 15s


scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']


  - job_name: 'dynamic-node-exporters'
    scrape_interval: 15s
    file_sd_configs:
      - files:
          - '/etc/prometheus/file_sd/node_targets.json'
        refresh_interval: 30s
```

This is an example structure. Preserve your existing jobs and append only the dynamic-node-exporters job.

What these settings mean:

- job_name: Identifies the monitoring job.
- files: Specifies where Prometheus reads target definitions.
- refresh_interval: Sets a periodic refresh interval of 30 seconds.
- scrape_interval: Tells Prometheus to scrape each discovered target every 15 seconds.

Prometheus also detects changes to configured files, so you normally do not need to restart it when only the target JSON file changes.



Prometheus

6. Validate the Prometheus configuration

If your Prometheus installation includes promtool, run:

sudo promtool check config /etc/prometheus/prometheus.yml

Expected output:

SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax

If promtool is not in your PATH, locate the binary:

command -v promtool

If your installation is manual, run the promtool binary from the same extracted Prometheus release directory.

7. Reload Prometheus

If your Prometheus service supports configuration reload, first check how it is configured. A common systemd setup uses the SIGHUP signal:

sudo systemctl kill -s HUP prometheus

This asks Prometheus to reload its configuration. Check the logs immediately afterward:

sudo journalctl -u prometheus -n 50 --no-pager

If you get a configuration reload error, correct the YAML or target file and try again.

If your installation uses a different service name or a manually launched process, adjust the commands to match your setup.

8. Verify discovered targets in Prometheus

Open the Prometheus web interface:

http://YOUR-PROMETHEUS-EC2-PUBLIC-IP:9090

For a private network, use the appropriate private IP or hostname.

Navigate to Status → Targets.

Find the job:

dynamic-node-exporters

You should see your configured EC2 instances listed as targets.

Expected result

Target 1

172.31.16.237:9100

UP

Target 2

172.31.18.138:9100

UP

The example assumes both exporters are running and reachable. If a target shows DOWN, verify that Node Exporter is running on port 9100 and that the target's EC2 security group allows TCP 9100 from the Prometheus server.

9. Test the dynamic discovery

Now add another target without modifying prometheus.yml.

Edit the JSON file:

sudo nano /etc/prometheus/file_sd/node_targets.json

Add another target group before the closing]:

```
,
{
  "targets": [
    "172.31.20.100:9100"
  ],
  "labels": {
    "environment": "uat",
    "team": "monitoring"
  }
}
```

Replace 172.31.20.100 with a real reachable Node Exporter address. Make sure the final JSON remains valid.

Validate it:

python3 -m json.tool /etc/prometheus/file_sd/node_targets.json

Wait for the file watcher or refresh interval, then revisit Status → Targets.

The new target should appear automatically without restarting Prometheus.

You can also run these PromQL queries in the Prometheus expression browser:

up{job="dynamic-node-exporters"}

To display targets by environment:

count by (environment) (up{job="dynamic-node-exporters"})

10. Optional: Discover EC2 instances directly

If your goal is to discover EC2 instances automatically based on AWS tags, Prometheus also supports native EC2 service discovery.



Prometheus

+1

For example, a separate job can use:

```
  - job_name: 'aws-ec2-node-exporters'
    ec2_sd_configs:
      - region: us-east-1
        port: 9100
        filters:
          - name: tag:Monitoring
            values: ['true']
          - name: instance-state-name
            values: ['running']
    relabel_configs:
      - source_labels: [__meta_ec2_private_ip]
        regex: (.+)
```

target_label: __address__

replacement: '${1}:9100'

Before using this configuration:

1. Tag the EC2 instances you want to monitor with Monitoring=true.
2. Give the Prometheus EC2 instance an IAM role with ec2:DescribeInstances permission.
3. Ensure the Node Exporter is installed and running on port 9100.
4. Allow TCP 9100 from the Prometheus instance in the target instances' security groups.
5. Validate the configuration and reload Prometheus.

This AWS discovery example is optional. Start with file-based discovery first, because it is simpler to troubleshoot.

Final verification checklist

Service discovery lab

0/7 completed

Prometheus is running

Dynamic target JSON file created

file_sd_configs added to prometheus.yml

Prometheus configuration validated

Configuration reloaded successfully

Targets visible under Status → Targets

New target discovered without a Prometheus restart

Official reference: Prometheus file-based service discovery guide.

8.Create alerting rules for monitoring disk space.

Create Prometheus Alerting Rules for Disk Space Monitoring

This lab shows you how to create disk-space alerting rules in Prometheus, verify them in the Prometheus UI, and connect Alertmanager to send notifications when an EC2 Linux server runs low on disk space.

Objective: Generate alerts when disk usage exceeds 80% or 90%, and verify that Prometheus evaluates the rules correctly.

1. Understand the monitoring architecture

EC2 Linux server

Node Exporter exposes filesystem metrics on port 9100

Prometheus — port 9090

Evaluates alerting rules periodically

Alertmanager — port 9093

Routes alerts to email or other receivers

2. Verify Node Exporter metrics

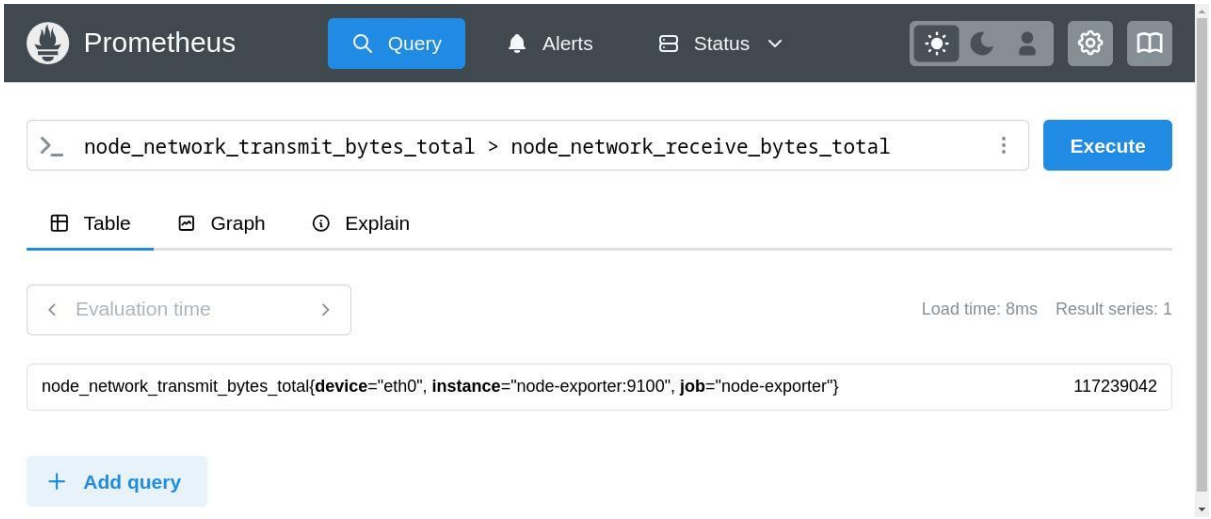
Log in to your Prometheus EC2 instance and run:

```
curl -sG http://localhost:9090/api/v1/query \
  --data-urlencode 'query=node_filesystem_avail_bytes'
```

If Prometheus is receiving filesystem metrics, the response will contain data for metrics such as node_filesystem_avail_bytes.

You can also open the Prometheus UI:

http://YOUR-PROMETHEUS-IP:9090



betterstack.com

In the Prometheus expression browser:

1. Enter node_filesystem_avail_bytes.
2. Click Execute.
3. Confirm that filesystem data appears.

If no data appears, navigate to Status → Targets and check whether your Node Exporter target is UP.

3. Create a directory for alert rules

Run these commands on the Prometheus server:

sudo mkdir -p /etc/prometheus/rules

Check your Prometheus service to identify its configuration file:

sudo systemctl cat prometheus

Look for the --config.file argument. The remaining commands assume the path is:

/etc/prometheus/prometheus.yml

If your installation uses a different path, replace it in the commands below.

4. Create the disk-space alert rules file

Create a new file:

sudo nano /etc/prometheus/rules/disk_alerts.yml

Paste the following configuration:

```
groups:
  - name: disk-space-alerts
    rules:
      - alert: DiskSpaceWarning
        expr: |
          100 * (
            1 -
            node_filesystem_avail_bytes{
              fstype!~"tmpfs|overlay|squashfs|ramfs",
              mountpoint!=""
            }
            /
            node_filesystem_size_bytes{
              fstype!~"tmpfs|overlay|squashfs|ramfs",
              mountpoint!=""
            }
          ) > 80
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Disk usage above 80%"
          description: "Disk usage on {{ $labels.instance }} at {{ $labels.mountpoint }} is {{ $value }}%."


      - alert: DiskSpaceCritical
```

```
    expr: |
      100 * (
        1 -
        node_filesystem_avail_bytes{
          fstype!~"tmpfs|overlay|squashfs|ramfs",
          mountpoint!=""
        }
        /
        node_filesystem_size_bytes{
          fstype!~"tmpfs|overlay|squashfs|ramfs",
          mountpoint!=""
        }
      ) > 90
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Disk usage above 90%"
      description: "Disk usage on {{ $labels.instance }} at {{ $labels.mountpoint }} is {{ $value }}%."
```

Save the file using Esc, type :wq, and press Enter.

These two rules monitor disk utilization:

Rule	Condition	Pending duration
DiskSpaceWarning	Usage above 80%	5 minutes
DiskSpaceCritical	Usage above 90%	2 minutes

The for field prevents brief spikes from immediately triggering alerts.

5. Configure Prometheus to load the rules

Open the main configuration file:

sudo nano /etc/prometheus/prometheus.yml

Add the following top-level section:

rule_files:

  - "/etc/prometheus/rules/disk_alerts.yml"

For example, your configuration should have this structure:

global:

  scrape_interval: 15s

  evaluation_interval: 15s

rule_files:

  - "/etc/prometheus/rules/disk_alerts.yml"

scrape_configs:

  - job_name: prometheus

    static_configs:

      - targets: ["localhost:9090"]

  # Preserve your existing exporter jobs here.

Important: If rule_files already exists, add the new filename to it. Do not duplicate the section or overwrite your existing scrape jobs.

6. Validate the rules and configuration

First, validate the rules:

sudo promtool check rules /etc/prometheus/rules/disk_alerts.yml

Expected output:

Checking /etc/prometheus/rules/disk_alerts.yml

SUCCESS: 2 rules found

Next, validate the main configuration:

sudo promtool check config /etc/prometheus/prometheus.yml

Resolve any YAML or configuration errors before reloading Prometheus.

7. Reload Prometheus

For a systemd-managed Prometheus service, run:

sudo systemctl kill -s HUP prometheus

Then check the service:

sudo systemctl status prometheus --no-pager

Inspect the latest logs:

sudo journalctl -u prometheus -n 30 --no-pager

Prometheus should remain active without configuration reload errors.

8. Verify the alert rules in the Prometheus UI

Open:

http://YOUR-PROMETHEUS-IP:9090

cnblogs.com

Navigate to Alerts or Status → Rules, depending on your Prometheus version.

Look for:

- DiskSpaceWarning
- DiskSpaceCritical

The alerts can appear in these states:

- Inactive: The condition is not currently true.
- Pending: The condition is true, but its required duration has not elapsed.
- Firing: The condition has remained true for the configured duration.

If both alerts are inactive, your server may simply have sufficient disk space. That is normal.

9. Test the disk-usage query

In the Prometheus expression browser, execute:

```
100 * (
 1 -
 node_filesystem_avail_bytes{
  fstype!~"tmpfs|overlay|squashfs|ramfs",
  mountpoint!=""
 }
 /
 node_filesystem_size_bytes{
  fstype!~"tmpfs|overlay|squashfs|ramfs",
  mountpoint!=""
 }
)
```

This returns disk usage percentages for matching filesystems.

To find filesystems exceeding 80% usage, execute:

```
(
 100 * (
  1 -
  node_filesystem_avail_bytes{
   fstype!~"tmpfs|overlay|squashfs|ramfs",
   mountpoint!=""
  }
  /
```

```
    node_filesystem_size_bytes{
      fstype!~"tmpfs|overlay|squashfs|ramfs",
      mountpoint!=""
    }
  )
) > 80
```

You can compare the result with the actual disk usage on your EC2 server:

```
df -h
```

You do not need to fill up your disk to test the rules. You can validate the rule syntax and query results without creating a real disk-space incident.

10. Connect Alertmanager for notifications

Prometheus evaluates the rules; Alertmanager handles notification delivery.

If Alertmanager is running on the same server, confirm that your Prometheus configuration includes:

```
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - "localhost:9093"
```

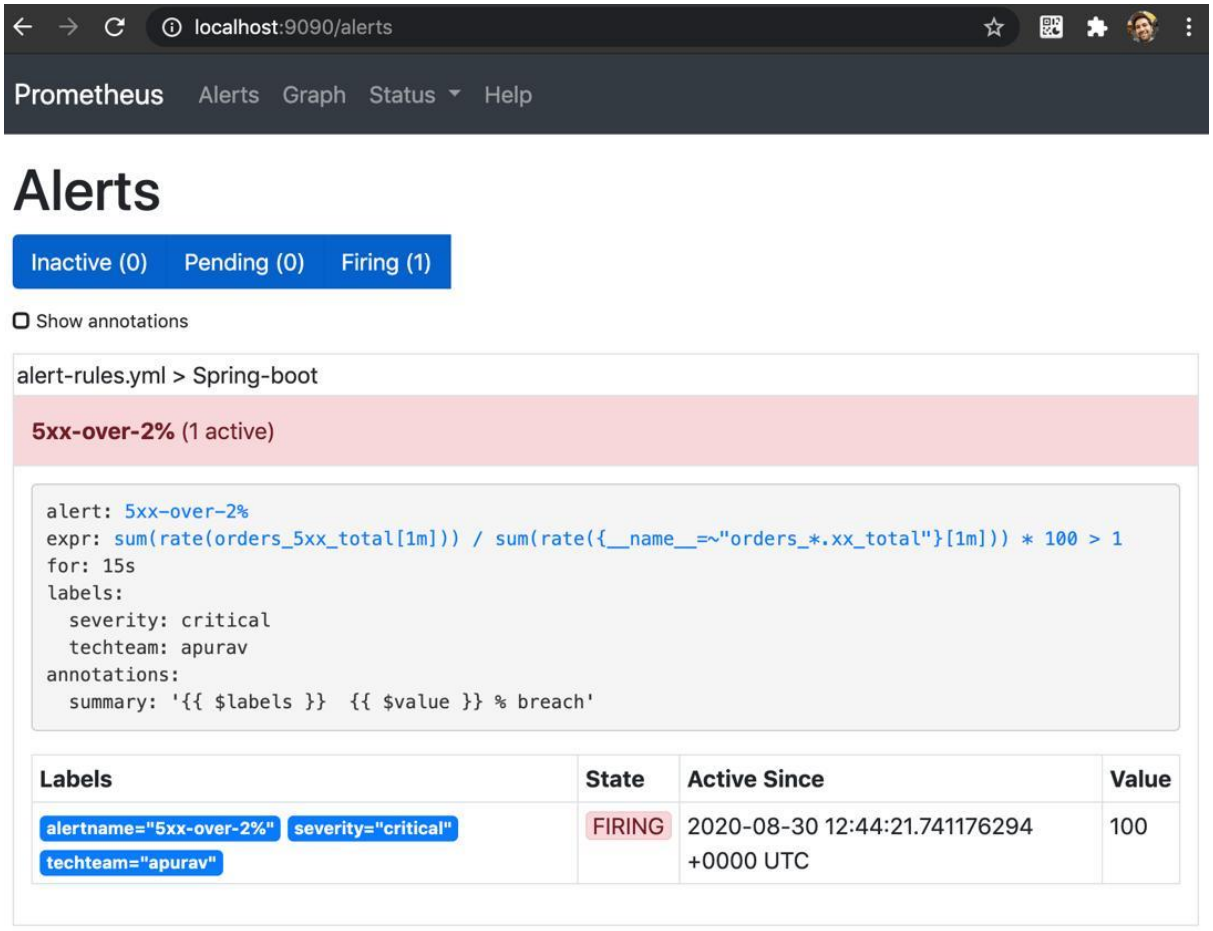
If Alertmanager runs on another EC2 instance, use its reachable private IP and port 9093.

Check its service:

```
sudo systemctl status alertmanager --no-pager
```

Open the Alertmanager UI:

http://YOUR-ALERTMANAGER-IP:9093



apuravchauhan.medium.com

Once the alert fires, it should appear in Alertmanager if the connection is configured correctly. To receive emails, you must also configure a valid SMTP receiver and routing rules in alertmanager.yml.

9.Create a custom Grafana dashboard using Prometheus metrics for specific insights.

Create a Custom Grafana Dashboard Using Prometheus Metrics

In this lab, you will create a custom Grafana dashboard to monitor your AWS EC2 Linux servers using Prometheus metrics. The dashboard will display CPU utilization, memory usage, disk space, and target health.

I can provide the step-by-step instructions and example dashboard screenshots to help you follow along. The screenshots below are illustrative references, not screenshots captured from your actual Grafana server.

1. Dashboard architecture

EC2 Linux servers

Node Exporter collects CPU, memory, and disk metrics

Prometheus — port 9090

Scrapes and stores metrics

Grafana — port 3000

Visualizes metrics in a custom dashboard

2. Open Grafana

Open your Grafana URL in your browser:

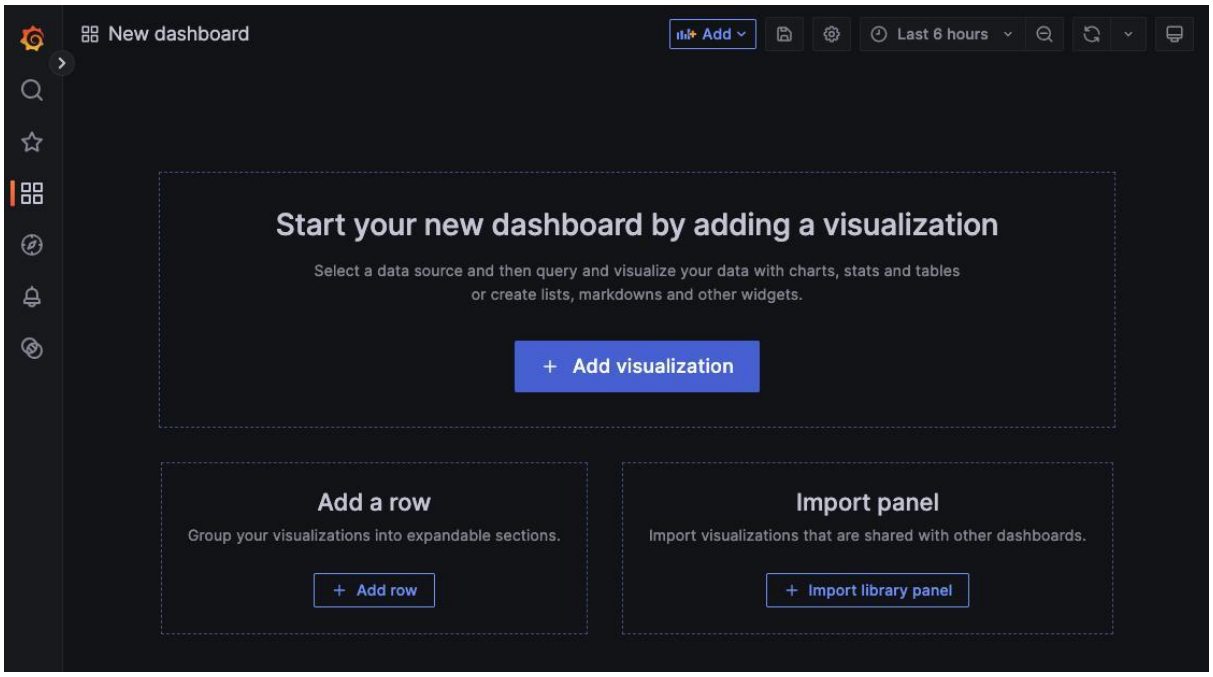
http://YOUR-GRAFANA-EC2-PUBLIC-IP:3000

Log in with your Grafana credentials.

Before proceeding, verify that you have already added Prometheus as a data source:

1. Open Connections → Data sources.
2. Select your Prometheus data source.
3. Confirm the connection test succeeds.

3. Create a new dashboard



metricfire.com

Follow these steps:

1. In the left navigation menu, select Dashboards.
2. Click New → New dashboard.
3. Click Add visualization.
4. Select your Prometheus data source.

You can now create panels that display your server metrics.

4. Create Panel 1 — CPU utilization



grafana.com

Set the panel title to CPU Usage (%).

Enter this PromQL query:

```
100 * (
  1 - avg by (instance) (
    rate(node_cpu_seconds_total{mode="idle"}[5m])
  )
)
```

Configure the visualization:

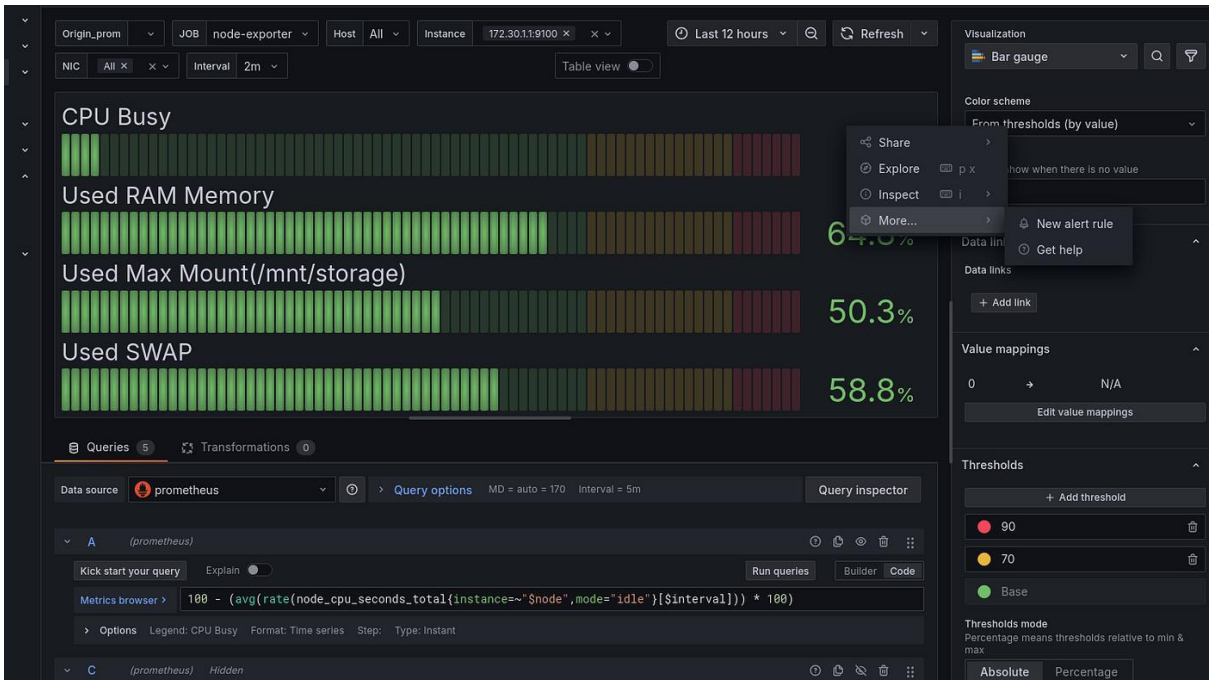
- Visualization: Time series
- Unit: Percent (0–100)
- Title: CPU Usage (%)

Click Run queries or Refresh if needed.

What this shows: The percentage of CPU time spent doing work rather than being idle, calculated per instance.

If the panel has no data, verify that Node Exporter is running and that Prometheus is scraping it successfully.

5. Create Panel 2 — Memory utilization



medium.com

Create another visualization and set the title to Memory Usage (%).

Use this query:

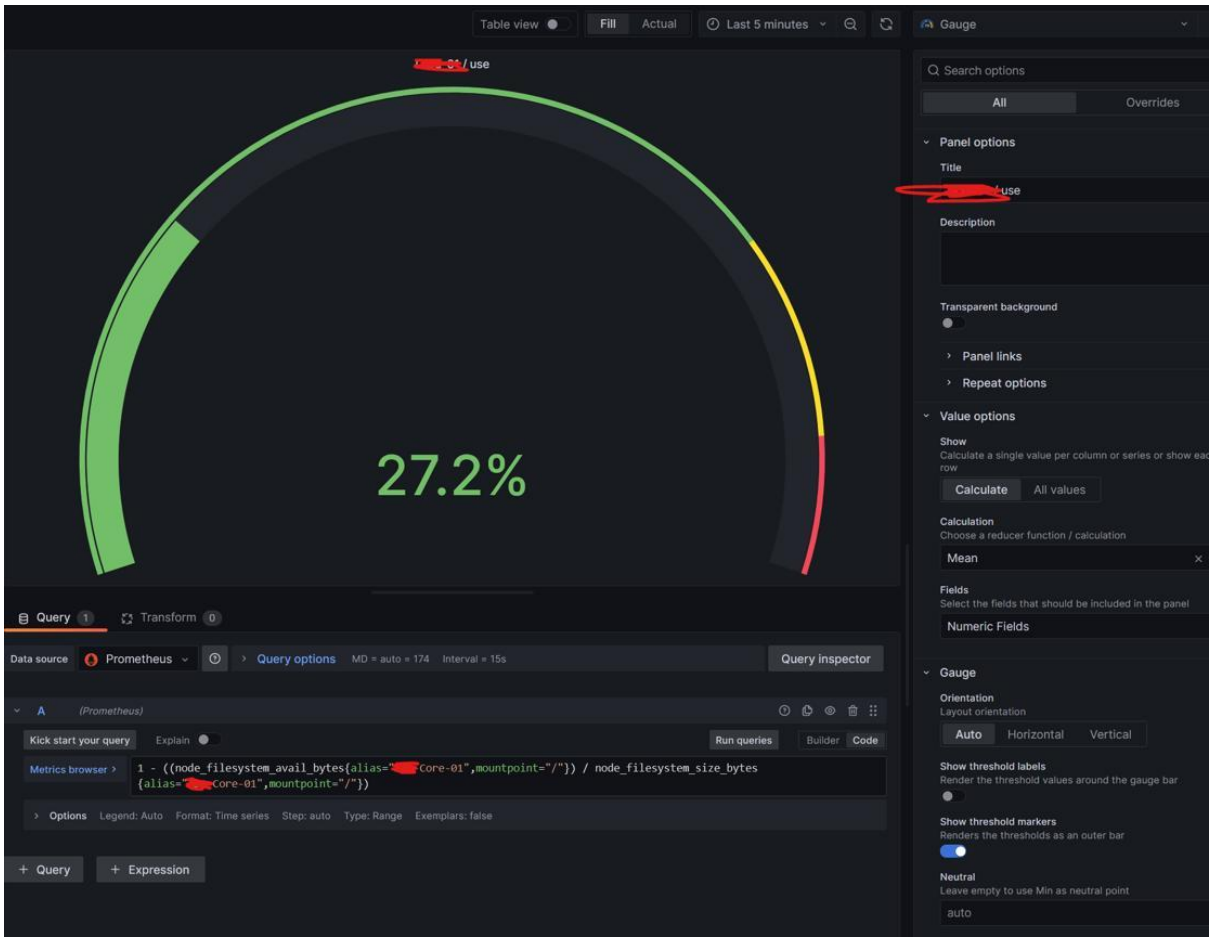
```
100 * (
  1 -
  node_memory_MemAvailable_bytes
  /
  node_memory_MemTotal_bytes
```

)

Set the visualization to Time series or Gauge, and choose the unit Percent (0–100).

What this shows: The percentage of total system memory that is not available for general use. MemAvailable is preferable to treating all Linux cache and buffers as unavailable memory.

6. Create Panel 3 — Disk usage



community.grafana.com

Set the panel title to Disk Usage by Mountpoint (%).

Use this query:

```
100 * (
  1 -
  node_filesystem_avail_bytes{
    fstype!~"tmpfs|overlay|squashfs|ramfs",
```

```
    mountpoint!=""
  }
  /
  node_filesystem_size_bytes{
    fstype!~"tmpfs|overlay|squashfs|ramfs",
    mountpoint!=""
  }
)
```

Recommended settings:

- Visualization: Time series or Bar gauge
- Unit: Percent (0–100)
- Legend: {{instance}} {{mountpoint}}

This lets you compare filesystem usage across your monitored EC2 instances.

For example, you can quickly identify a root filesystem approaching 90% usage.

7. Create Panel 4 — Available memory

Set the panel title to Available Memory.

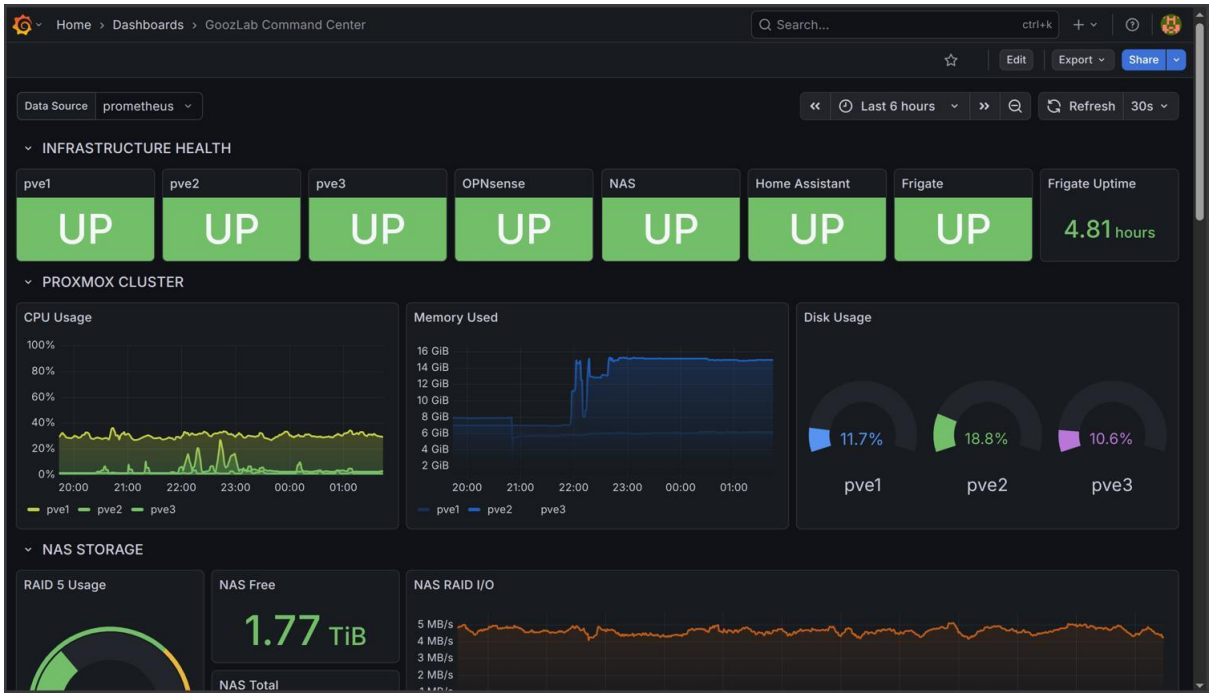
Use this query:

node_memory_MemAvailable_bytes / 1024^3

Set the visualization to Time series and the unit to Gibibytes (GiB).

This panel shows how much memory remains available on each server.

8. Create Panel 5 — Prometheus target health



sohrabros.github.io

Set the panel title to Node Exporter Targets UP.

Use this query:

sum(up{job=~".*node.*"})

Choose Stat as the visualization.

This query counts the targets whose job names contain node and whose up value is 1. Adjust the job selector to match your actual Prometheus job name.

For per-target health, use:

up

This displays individual targets, with 1 indicating a successful scrape and 0 indicating a failed scrape.

9. Arrange your dashboard panels

Your dashboard can be arranged as follows:

EC2 System Monitoring

CPU Usage

37%

Example value

Memory Usage

62%

Example value

Disk Usage

68%

Example value

Targets UP

3 / 3

Example value

CPU Usage (%) — example trend

Illustrative chart only; not live Prometheus data.

Disk Usage by Mountpoint (%)

/

68%

Illustrative value for a root filesystem.

The layout above is a dashboard mockup to help you plan your panels. The real dashboard values will come from your Prometheus data source.

10. Save the dashboard

Once you have created all five panels:

1. Click Apply to save each panel to the dashboard.
2. Click Save dashboard.
3. Enter the dashboard name:

EC2 System Monitoring

4. Click Save again.
5. Select a suitable time range, such as Last 1 hour.
6. Enable auto-refresh, for example 30 seconds, if desired.

11. Verify your dashboard

Confirm that:

- CPU utilization displays data.
- Memory usage and available memory display data.
- Disk usage is visible for your monitored filesystems.
- Node Exporter targets are showing their actual health.
- The dashboard refreshes as new metrics arrive.

If a panel displays No data, open Prometheus and check Status → Targets. Make sure the Node Exporter targets are UP and that the relevant metric names exist.